Enerji israf eden 113 kod kalıbı, ölçümleriyle

Yeşil Kodlama (Green Coding) ve Dijital Karbon Ayak İzi Optimizasyon Aracı

Sakarya Üniversitesi · Yunus Emre Sevindik · Mehmet Zahid Görgeç

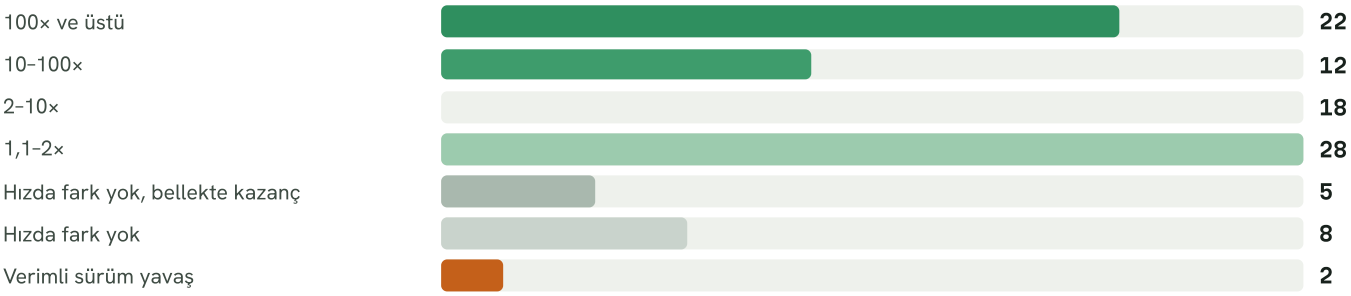
ÖZET

Yeşil Kod'un aradığı **113 kalıbın** her biri için bir israflı ve bir verimli örnek yazıldı. İkisi aynı girdiyle çalıştırıldı, çıktıları karşılaştırıldı, süreleri ve bellek kullanımları ölçüldü. **95 kural ölçüldü**; çıktısı karşılaştırılabilen 94 kuralın hepsinde iki sürüm birebir aynı sonucu verdi, 1 kuralın çıktısı rastgele. 18 kural veritabanı sunucusu, GPU ya da ağ gerektirdiği için ölçülmedi; nedenleri yazılı. **11 kalıbı** motor kendisi düzeltir; onların düzeltmesi de aynı yöntemle sınandı.

2,83×

ölçülen kazancın ortancası

ÖLÇÜLEN KAZANÇ (israflı sürümün süresi ÷ verimli sürümün süresi)



Nasıl okunur?

- **Önem**: *yüksek* veri büyüdükçe katlanan israf, *orta* belirgin israf, *bilgi* küçük ama kolay kazanç.
- **Motor düzeltir**: Yeşil Kod bu kalıbı kodda kendisi yeniden yazar; diğerleri için öneri verir.
- **Kazanç**: israflı sürümün süresinin verimli sürümün süresine oranı (5 turun ortancası). Bekleme kurallarında süre aynıdır; oran işlemci süresinden alınır.
- **Fark yok** ya da **yavaş** çıkan sonuçlar da yazıldı; nedeni her kuralın altındaki notta.
- Kodlarda vurgulu satırlar iki sürüm arasında değişen satırlardır.

Ölçüm: Apple M4, Python 3.13.9, 29 Ekim 2026. Enerji tahmini işlemci süresiyle orantılıdır; aynı işlemcide oran enerji oranına eşittir.

Her kuralı telefonunda aç, kendi cihazında ölç.

yes3760.github.io/yesil-kod/#kurallar

Gruplar

01	Veritabanı	14	02 Döngü ve algoritma	18	03 Veri yapıları ve Python dili	31
04	Metin ve düzenli ifadeler	7	05 Veri bilimi (pandas, NumPy)	17	06 Makine öğrenmesi	4
07	Ağ ve bulut	7	08 Dosya ve G/Ç	8	09 Eşzamanlılık ve bekleme	7

01 Veritabanı

14 kural

YK001 yüksek Motor düzeltir

1,86× hızlı

Döngü içinde veritabanı sorgusu (N+1)

Her döngü adımı veritabanına ayrı bir istek gönderiyor. Verinin tamamını tek bir JOIN ya da 'WHERE id IN (...)' sorgusuyla çekin.

NEDEN ENERJİ HARCAR?

Her sorgu bir ağ gidiş-dönüşü, sorgu ayrıştırma ve disk erişimi demektir. 1.000 satır için 1.000 ayrı sorgu, aynı veriyi tek sorguda çekmekten çok daha fazla işlemci, ağ ve bekleme süresi harcar.

NE ZAMAN SORUN DEĞİL?

Döngü yalnızca birkaç adım dönüyorsa ya da her sorgu farklı bir veritabanına gidiyorsa fark küçüktür.

YAPILAN İŞ **n sorgu → 1 sorgu**

İSRAFLI

```
def siparis_raporu(db, siparisler):
    rapor = []
    for s in siparisler:
        ad = db.execute("SELECT ad FROM musteri WHERE id = ?",
  (s["musteri_id"],)).fetchone()[0]
        rapor.append((s["id"], ad))
    return rapor
```

VERİMLİ

```
def siparis_raporu(db, siparisler):
    kimlikler = sorted({s["musteri_id"] for s in siparisler})
    yer_tutucular = ", ".join("?" * len(kimlikler))
    sorgu = "SELECT id, ad FROM musteri WHERE id IN (" +
  yer_tutucular + ")"
    adlar = dict(db.execute(sorgu, kimlikler).fetchall())
    return [(s["id"], adlar[s["musteri_id"]]) for s in
  siparisler]
```

ÖLÇÜM süre 2,1 ms → 1,1 ms · bellek 121,4 KB → 280,5 KB · aynı sonuç ✓

Bellekteki SQLite'ta ölçüldü. Ağ üzerindeki bir veritabanında her sorgu ayrıca bir gidiş-dönüş (çoğunlukla 0,2–1 ms) ekler; fark çok daha büyür. Motorun düzeltmesi de aynı sonucu verdi (1,42×).

Kaynak: Green Software Foundation: Keep request counts low · Django: select_related · docs.djangoproject.com/en/stable/topics/db/optimization/

YK021 bilgi Öneri verir

3,80× hızlı

SELECT * ile gereksiz sütun çekme

SELECT * kullanılmayan sütunları da diskten okur ve ağdan taşır. Yalnızca gereken sütunları yazın.

NEDEN ENERJİ HARCAR?

Kullanılmayan sütunlar da diskten okunur, belleğe alınır ve ağdan taşınır; geniş tablolarda bu, gereken verinin katları kadar G/Ç demektir. Kapsayan (covering) dizin kullanımını da engeller.

NE ZAMAN SORUN DEĞİL?

Bütün sütunlar gerçekten kullanılıyorsa ya da tablo çok darsa fark küçüktür.

YAPILAN İŞ **bütün sütunlar → gereken sütunlar**

İSRAFLI

```
def musteri_adlari(db):
    return [r[1] for r in db.execute("SELECT * FROM musteri")]
```

VERİMLİ

```
def musteri_adlari(db):
    return [r[0] for r in db.execute("SELECT ad FROM musteri")]
```

ÖLÇÜM süre 2,8 ms → 736 µs · bellek 150,6 KB → 148,6 KB · aynı sonuç ✓

Kaynak: Green Software Foundation: Reduce transmitted data · Django: only()/values() · patterns.greensoftware.foundation/development

---

YK022 orta Öneri verir

1.201× hızlı

Saymak için bütün satırları çekme

len(...fetchall()) bütün satırları veritabanından taşır. SELECT COUNT(*) ile sayıyı veritabanına hesaplatın.

NEDEN ENERJİ HARCAR?

Saymak için bütün satırları çekmek, onları ağdan taşımayı ve Python nesnesine çevirmeyi gerektirir. COUNT(*) yalnızca bir sayı döndürür ve çoğu zaman dizinden hesaplanır.

NE ZAMAN SORUN DEĞİL?

Satırlar zaten başka bir iş için çekiliyorsa len() ek maliyet getirmez.

YAPILAN İŞ **n satır taşıma → 1 sayı**

İSRAFLI

```
def siparis_sayisi(db):
    return len(db.execute("SELECT id FROM siparis").fetchall())
```

VERİMLİ

```
def siparis_sayisi(db):
    return db.execute("SELECT COUNT(*) FROM
  siparis").fetchone()[0]
```

ÖLÇÜM süre 5,1 ms → 4,3 µs · bellek 1,6 MB → 673 B · aynı sonuç ✓

Kaynak: Django: Database access optimization (count) · docs.djangoproject.com/en/stable/ref/models/querysets/#count

---

YK023 orta Öneri verir **1,91× hızlı**

Veritabanı yerine Python'da filtreleme

Bütün satırları çekip Python'da elemek gereksiz veri taşır. Koşulu sorgunun WHERE kısmına yazın; veritabanı dizinleri kullanır.

NEDEN ENERJİ HARCAR?

Gereksiz satırlar diskten okunur, ağdan taşınır ve Python'da elenir. WHERE ile veritabanı dizinleri kullanarak yalnızca gerekenleri döndürür.

NE ZAMAN SORUN DEĞİL?

Koşul SQL ile ifade edilemiyorsa (karmaşık Python mantığı) Python'da filtrelemek gerekebilir.

YAPILAN İŞ **n satır → eşleşenler**

İSRAFLI

```
def buyuk_siparisler(db):
    return [s for s in db.execute("SELECT id, tutar FROM
  siparis").fetchall() if s[1] > 500]
```

VERİMLİ

```
def buyuk_siparisler(db):
    return db.execute("SELECT id, tutar FROM siparis WHERE
  tutar > ?", (500,)).fetchall()
```

ÖLÇÜM süre 8,1 ms → 4,2 ms · bellek 2,3 MB → 1,1 MB · aynı sonuç ✓

Kaynak: Green Software Foundation: Reduce transmitted data · Django: filter() · `docs.djangoproject.com/en/stable/topics/db/optimization/`

YK031 yüksek Öneri verir **143× hızlı**

Döngüde commit

Her commit işlem kaydını diske eşitler (fsync); döngüde commit yüzlerce kez disk beklemesi demektir. Döngü bittikten sonra bir kez commit edin.

NEDEN ENERJİ HARCAR?

Her commit, işlem kaydının diske kalıcı yazılmasını (fsync) bekler; diskin fiziksel yazma süresi döngü adımı sayısıyla çarpılır. SQLite SSS'si bunu INSERT'lerin yavaşlığının ana nedeni olarak açıklar.

NE ZAMAN SORUN DEĞİL?

Her kaydın tek tek kalıcı olması iş kuralıysa (ödeme) ayrı commit gerekir.

YAPILAN İŞ **n disk eşitleme → 1**

İSRAFLI

```
def kayitlari_ekle(baglanti, kayitlar):
    for k in kayitlar:
        baglanti.execute("INSERT INTO log (mesaj) VALUES (?)",
  (k,))
        baglanti.commit()
```

VERİMLİ

```
def kayitlari_ekle(baglanti, kayitlar):
    with baglanti:
        baglanti.executemany("INSERT INTO log (mesaj) VALUES
  (?)", [(k,) for k in kayitlar])
```

ÖLÇÜM süre 60,8 ms → 425 µs · bellek 19,8 KB → 4,0 KB · aynı sonuç ✓

Diskteki bir SQLite dosyasıyla ölçüldü.

Kaynak: SQLite SSS: INSERT neden yavaş (işlem başına disk eşitleme) · `www.sqlite.org/faq.html#q19`

YK032 yüksek Öneri verir **1,57× hızlı**

Döngüde tek tek INSERT, UPDATE ya da DELETE

Her satır için ayrı yazma komutu veritabanına ayrı bir gidiş-dönüş yapar. executemany() ya da toplu ekleme kullanın.

NEDEN ENERJİ HARCAR?

Her execute ayrı bir gidiş-dönüş, ayrıştırma ve plan adımıdır; executemany aynı hazırlanmış komutu toplu çalıştırır.

NE ZAMAN SORUN DEĞİL?

Satır sayısı çok küçükse fark önemsizdir.

YAPILAN İŞ **n gidiş–dönüş → 1**

İSRAFLI

```
def fiyat_guncelle(db, urunler):
    for u in urunler:
        db.execute("UPDATE urun SET fiyat = ? WHERE id = ?",
  (u["fiyat"], u["id"]))
```

VERİMLİ

```
def fiyat_guncelle(db, urunler):
    db.executemany("UPDATE urun SET fiyat = ? WHERE id = ?",
  [(u["fiyat"], u["id"]) for u in urunler])
```

ÖLÇÜM süre 2,8 ms → 1,8 ms · bellek 18,1 KB → 80,7 KB · aynı sonuç ✓

Kaynak: PEP 249 (DB-API): executemany · `peps.python.org/pep-0249/#executemany`

YK033 yüksek Öneri verir ölçülmedi

Döngüde objects.create() (Django)

Her create() ayrı bir INSERT sorgusudur. Nesneleri listede toplayıp bulk_create() ile tek sorguda ekleyin.

NEDEN ENERJİ HARCAR?
Her create() ayrı bir INSERT ve çoğu zaman ayrı bir işlemdir; bulk_create tek sorguda toplu ekler.

NE ZAMAN SORUN DEĞİL?
post_save sinyalleri ya da özel save() mantığı gerekiyorsa bulk_create bunları çalıştırmaz.

YAPILAN İŞ **n sorgu → 1**

İSRAFLI

```
def ogrenci_kaydet(satirlar):
    for s in satirlar:
        Ogrenci.objects.create(ad=s["ad"], numara=s["no"])
```

VERİMLİ

```
def ogrenci_kaydet(satirlar):
    Ogrenci.objects.bulk_create([Ogrenci(ad=s["ad"],
  numara=s["no"]) for s in satirlar])
```

ÖLÇÜLMEDİ Django projesi ve veritabanı sunucusu gerektirir. Kazanç sorgu sayısındadır: n INSERT → 1.

Kaynak: Django belgeleri: bulk_create · docs.djangoproject.com/en/stable/ref/models/querysets/#bulk-create

---

YK034 orta Öneri verir ölçülmedi

Döngüde save()

Döngüde her nesneyi ayrı save() etmek ayrı bir UPDATE sorgusudur. bulk_update() ya da queryset.update() kullanın.

NEDEN ENERJİ HARCAR?
Her save() bütün alanları içeren ayrı bir UPDATE gönderir.

NE ZAMAN SORUN DEĞİL?
save() içinde özel iş mantığı ya da sinyaller varsa toplu güncelleme bunları atlar.

YAPILAN İŞ **n sorgu → 1**

İSRAFLI

```
def notlari_yukselt(ogrenciler):
    for o in ogrenciler:
        o.not_ortalamasi += 5
        o.save()
```

VERİMLİ

```
from django.db.models import F


def notlari_yukselt(ogrenciler):
    ogrenciler.update(not_ortalamasi=F("not_ortalamasi") + 5)
```

ÖLÇÜLMEDİ Django projesi ve veritabanı sunucusu gerektirir. Kazanç sorgu sayısındadır: n UPDATE → 1.

Kaynak: Django belgeleri: bulk_update · update() · docs.djangoproject.com/en/stable/ref/models/querysets/#bulk-update

---

YK035 orta Öneri verir ölçülmedi

Saymak için sorgu sonucunu len() ile ölçme

len(queryset) bütün nesneleri yükler. Yalnızca sayı gerekiyorsa .count() kullanın; veritabanı COUNT(*) çalıştırır.

NEDEN ENERJİ HARCAR?
len() sorgu sonucunu bütünüyle çekip her satır için bir model nesnesi oluşturur; count() yalnızca bir sayı ister.

NE ZAMAN SORUN DEĞİL?
Nesneler zaten sonra kullanılacaksa len() fazladan sorgu atmaz.

YAPILAN İŞ **n nesne → 1 sayı**

İSRAFLI

```
def aktif_kullanici_sayisi():
    return len(Kullanici.objects.filter(aktif=True))
```

VERİMLİ

```
def aktif_kullanici_sayisi():
    return Kullanici.objects.filter(aktif=True).count()
```

ÖLÇÜLMEDİ Django projesi ve veritabanı sunucusu gerektirir. Kazanç taşınan veridedir: n nesne → 1 sayı.

Kaynak: Django belgeleri: count() ve len() · docs.djangoproject.com/en/stable/ref/models/querysets/#count

---

YK036 bilgi Öneri verir — fark yok

Tek satır için LIMIT olmadan sorgu

WHERE ve LIMIT olmadan yazılan sorgu bütün tabloyu döndürmeye hazırlanır, fetchone() yalnızca ilkini kullanır. Sorguya LIMIT 1 ekleyin.

NEDEN ENERJİ HARCAR?
WHERE ve LIMIT olmadan veritabanı bütün sonucu üretmeye hazırlanır; LIMIT 1 ilk satırda durmasına izin verir.

NE ZAMAN SORUN DEĞİL?
Tablo tek satırlıksa fark yoktur.

YAPILAN İŞ **bütün sonuç → ilk satır**

İSRAFLI

```
def ilk_ayar(db):
    return db.execute("SELECT anahtar, deger FROM
ayar").fetchone()
```

VERİMLİ

```
def ilk_ayar(db):
    return db.execute("SELECT anahtar, deger FROM ayar LIMIT
1").fetchone()
```

ÖLÇÜM süre 1,4 µs → 1,3 µs · bellek 718 B → 718 B · aynı sonuç ✓

SQLite imleci zaten ilk satırda durur; bu yüzden fark yok denecek kadar azdır. Fark, sonucu önceden hazırlayıp ağdan gönderen sunucu veritabanlarında görülür.

Kaynak: SQL: LIMIT ile erken durma · www.sqlite.org/lang_select.html#limitoffset

---

YK037 orta Öneri verir — 1,57× hızlı

ORDER BY RANDOM() ile rastgele satır

ORDER BY RANDOM() / RAND() bütün tabloyu okuyup sıralar. Rastgele bir kimlik seçip o satırı çekin ya da TABLESAMPLE kullanın.

NEDEN ENERJİ HARCAR?
Her satıra rastgele bir sayı atanır ve bütün tablo sıralanır; tek satır için tablo boyutu kadar iş yapılır.

NE ZAMAN SORUN DEĞİL?
Tablo çok küçükse sorun değildir.

YAPILAN İŞ **O(n log n) → O(1)**

İSRAFLI

```
def rastgele_soru(db):
    return db.execute("SELECT * FROM soru ORDER BY RANDOM()
LIMIT 1").fetchone()
```

VERİMLİ

```
import random


def rastgele_soru(db):
    en_kucuk, en_buyuk = db.execute("SELECT MIN(id), MAX(id)
FROM soru").fetchone()
    secilen = random.randint(en_kucuk, en_buyuk)
    return db.execute("SELECT id, metin FROM soru WHERE id >= ?
ORDER BY id LIMIT 1", (secilen,)).fetchone()
```

ÖLÇÜM süre 1,1 ms → 721 µs · bellek 838 B → 1,1 KB · çıktı rastgele

Sonuç rastgele olduğu için çıktılar karşılaştırılmadı; yalnızca süre ölçüldü.

Kaynak: PostgreSQL belgeleri: TABLESAMPLE · www.postgresql.org/docs/current/sql-select.html#SQL-FROM

YK038 bilgi Öneri verir — 2,87× hızlı

Başında joker olan LIKE araması

LIKE '%...' dizini kullanamaz ve bütün tabloyu tarar. Metin araması için tam metin dizini (full-text) kullanın.

NEDEN ENERJİ HARCAR?
B-ağacı dizini önek araması yapabilir; '%' ile başlayan desen dizini kullanamaz ve bütün satırları tarar.

NE ZAMAN SORUN DEĞİL?
Tablo küçükse ya da sorgu nadiren çalışıyorsa kabul edilebilir.

YAPILAN İŞ **O(log n) → O(n)**

İSRAFLI

```
def ada_gore_ara(db, parca):
    return db.execute("SELECT id, ad FROM musteri WHERE ad LIKE
'%' || ? || '%'", (parca,)).fetchall()
```

VERİMLİ

```
def ada_gore_ara(db, parca):
    # musteri_ara: CREATE VIRTUAL TABLE musteri_ara USING
fts5(ad, tokenize='trigram')
    ifade = '"' + parca.replace('"', '""') + '"'
    return db.execute("SELECT rowid, ad FROM musteri_ara WHERE
musteri_ara MATCH ? ORDER BY rowid", (ifade,)).fetchall()
```

ÖLÇÜM süre 3,0 ms → 1,1 ms · bellek 115,0 KB → 115,1 KB · aynı sonuç ✓

İyi sürüm FTS5 trigram dizinini kullanır; aynı alt metin aramasını yapar ve aynı satırları döndürür. Dizinin kurulumu ölçüme dahil değildir.

Kaynak: Veritabanı dizinleri: B-tree önek araması · www.sqlite.org/optoverview.html#the_like_optimization

---

YK039 orta Öneri verir **8,89× hızlı**

Toplamı Python'da hesaplama

Bütün satırları çekip sum/min/max'ı Python'da hesaplamak gereksiz veri taşır. SUM(), MIN(), MAX() ile veritabanına hesaplatın.

NEDEN ENERJİ HARCAR?

Toplam için bütün satırları taşımak yerine SUM() sonucu veritabanında hesaplar ve tek değer döndürür.

NE ZAMAN SORUN DEĞİL?

Satırlar zaten başka işler için çekiliyorsa ek maliyet yoktur.

YAPILAN İŞ **n satır → 1 değer**

İSRAFLI

```
def toplam_ciro(db):
    return sum(satir[0] for satir in db.execute("SELECT tutar
  FROM siparis"))
```

VERİMLİ

```
def toplam_ciro(db):
    return db.execute("SELECT SUM(tutar) FROM
  siparis").fetchone()[0]
```

ÖLÇÜM süre 5,5 ms → 617 µs · bellek 1.014 B → 675 B · aynı sonuç ✓

Kaynak: Django belgeleri: aggregate() · `docs.djangoproject.com/en/stable/topics/db/aggregation/`

YK040 orta Öneri verir **1,89× hızlı**

Biçimlendirilmiş SQL sorgusu

f-string ya da format ile kurulan sorgu her seferinde farklı metin olur; veritabanı sorgu planını yeniden kullanamaz ve SQL enjeksiyonuna açık kalır. Parametreli sorgu kullanın.

NEDEN ENERJİ HARCAR?

Her farklı sorgu metni yeniden ayrıştırılır ve planlanır; parametreli sorguda metin aynı kaldığı için hazırlanmış plan yeniden kullanılır. Ayrıca SQL enjeksiyonunu önler.

NE ZAMAN SORUN DEĞİL?

Tablo ya da sütun adı gibi parametreyle verilemeyen kısımlar için biçimlendirme gerekebilir; bunlar güvenilir kaynaktan gelmelidir.

YAPILAN İŞ **n plan → 1 plan**

İSRAFLI

```
def musteri_bul(db, sehir):
    return db.execute(f"SELECT id, ad FROM musteri WHERE sehir
  = '{sehir}'").fetchall()
```

VERİMLİ

```
def musteri_bul(db, sehir):
    return db.execute("SELECT id, ad FROM musteri WHERE sehir =
  ?", (sehir,)).fetchall()
```

ÖLÇÜM süre 3,1 ms → 1,6 ms · bellek 274,8 KB → 231,1 KB · aynı sonuç ✓

300 farklı değerle ölçüldü. Python'un sqlite3 modülü son 128 sorgu metnini önbellekte tutar; her değer ayrı metin olunca önbellek yetmez ve sorgu her seferinde yeniden derlenir.

Kaynak: PEP 249 (DB-API): parametre işaretleri · `peps.python.org/pep-0249/#paramstyle`

---

02 Döngü ve algoritma

18 kural

YK002 yüksek Motor düzeltir

375× hızlı

Döngü içinde listede arama

Listede 'in' araması her seferinde listeyi baştan sona tarar. Döngüden önce listeyi set() ile kümeye çevirin; arama sabit sürede yapılır.

NEDEN ENERJİ HARCAR?

Listede 'in' araması elemanları baştan sona tek tek karşılaştırır (O(n)); döngü içinde bu n × m karşılaştırma eder. Küme (set) karma tablosu kullanır ve aramayı sabit sürede yapar.

NE ZAMAN SORUN DEĞİL?

Liste birkaç elemanlıksa ya da arama yalnızca bir kez yapılıyorsa kümeye çevirmeye değmez.

YAPILAN İŞ **O(n·m) → O(n + m)**

İSRAFLI

```
def yoklama(gelenler, kayit):
    kayitlilar = list(kayit)
    return [no for no in gelenler if no in kayitlilar]
```

VERİMLİ

```
def yoklama(gelenler, kayit):
    kayitlilar = set(kayit)
    return [no for no in gelenler if no in kayitlilar]
```

ÖLÇÜM süre 23,2 ms → 62 µs · bellek 32,1 KB → 160,3 KB · aynı sonuç ✓

Motorun düzeltmesi de aynı sonucu verdi (321×).

Kaynak: Python wiki: TimeComplexity · `wiki.python.org/moin/TimeComplexity`

YK003 orta Motor düzeltir

109× hızlı

Döngüde değişmeyen işi tekrar yapma

Bu çağrının sonucu döngü boyunca değişmiyor. Döngüden önce bir kez yapın ve sonucu bir değişkende tutun.

NEDEN ENERJİ HARCAR?

Döngüde değişmeyen bir çağrı (JSON ayrıştırma, regex derleme) her adımda aynı sonucu yeniden üretir; iş döngü uzunluğu kadar katlanır.

NE ZAMAN SORUN DEĞİL?

Çağrının yan etkisi varsa ya da her adımda farklı sonuç vermesi gerekiyorsa (ör. dosya her adımda değişiyorsa) dışarı taşınmamalıdır.

YAPILAN İŞ **n iş → 1 iş**

İSRAFLI

```
import json


def fiyatla(urunler, ayar_metni):
    sonuc = []
    for u in urunler:
        ayar = json.loads(ayar_metni)
        sonuc.append(u["fiyat"] * (1 + ayar["kdv"]))
    return sonuc
```

VERİMLİ

```
import json


def fiyatla(urunler, ayar_metni):
    kdv = json.loads(ayar_metni)["kdv"]
    return [u["fiyat"] * (1 + kdv) for u in urunler]
```

ÖLÇÜM süre 18,1 ms → 166 µs · bellek 101,8 KB → 93,6 KB · aynı sonuç ✓

Motorun düzeltmesi de aynı sonucu verdi (83,9×).

Kaynak: Python wiki: PerformanceTips · `wiki.python.org/moin/PythonSpeed/PerformanceTips`

YK004 yüksek Motor düzeltir **1.681× hızlı**

Önbelleksiz çift özyineleme

Fonksiyon kendini birden fazla kez çağırıyor; aynı ara sonuçlar katlanarak yeniden hesaplanır. @functools.lru_cache ekleyin ya da döngüye çevirin.

NEDEN ENERJİ HARCAR?

f(n-1) + f(n-2) gibi özyineleme aynı alt problemleri tekrar tekrar çözer; çağrı sayısı üstel büyür. Önbellek her alt sonucu bir kez hesaplar.

NE ZAMAN SORUN DEĞİL?

Fonksiyonun yan etkisi varsa (yazdırma, çizim) ya da argümanları karma yapılamıyorsa (liste, sözlük) önbellek kullanılamaz.

YAPILAN İŞ `O(2ⁿ) → O(n)`

İSRAFLI

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

VERİMLİ

```
from functools import lru_cache


@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

ÖLÇÜM süre 5,5 ms → 3,3 µs · bellek 224 B → 1,9 KB · aynı sonuç ✓

Önbellek her çağrıdan önce temizlendi; iyi sürüm de her seferinde sıfırdan hesapladı.

Motorun düzeltmesi de aynı sonucu verdi (1.683×).

Kaynak: Python belgeleri: functools.lru_cache · `docs.python.org/3/library/functools.html#functools.lru_cache`

YK006 bilgi Öneri verir **158× hızlı**

İç içe döngü (O(n²))

İç içe döngüler veri büyüdükçe karesel olarak yavaşlar. İç döngü bir eşleştirme yapıyorsa sözlük (dict) ile tek geçişe indirin.

NEDEN ENERJİ HARCAR?

İç içe iki döngü n × m adım atar; veri on kat büyüyünce iş yüz kat büyür. Eşleştirme yapan iç döngü sözlükle tek geçişe indirilebilir.

NE ZAMAN SORUN DEĞİL?

Gerçekten bütün çiftleri karşılaştırmak gerekiyorsa (ör. mesafe matrisi) iç içe döngü doğaldır.

YAPILAN İŞ `O(n·m) → O(n + m)`

İSRAFLI

```
def eslestir(ogrenciler, dersler):
    eslesme = []
    for o in ogrenciler:
        for d in dersler:
            if d["ogrenci_id"] == o["id"]:
                eslesme.append((o["ad"], d["ad"]))
    return eslesme
```

VERİMLİ

```
from collections import defaultdict


def eslestir(ogrenciler, dersler):
    ogrencinin = defaultdict(list)
    for d in dersler:
        ogrencinin[d["ogrenci_id"]].append(d["ad"])
    return [(o["ad"], ders) for o in ogrenciler for ders in
  ogrencinin.get(o["id"], ())]
```

ÖLÇÜM süre 40,9 ms → 259 µs · bellek 14,1 KB → 86,9 KB · aynı sonuç ✓

Kaynak: Python wiki: TimeComplexity · `wiki.python.org/moin/TimeComplexity`

YK011 orta Motor düzeltir **4,26× hızlı**

En küçük ya da en büyük için sıralama

Bütün listeyi sıralayıp ilk ya da son elemanı almak O(n log n) iştir. min() ve max() aynı sonucu tek geçişte (O(n)) bulur.

NEDEN ENERJİ HARCAR?

Sıralama O(n log n) iş ve tam bir kopya demektir; en küçük ya da en büyük eleman için tek geçiş (O(n)) yeterlidir.

NE ZAMAN SORUN DEĞİL?

Sıralı listenin tamamı başka bir yerde de kullanılıyorsa sıralama boşa gitmez.

YAPILAN İŞ `O(n log n) → O(n)`

İSRAFLI

```
def en_ucuz_urun(urunler):
    return sorted(urunler, key=lambda u: u["fiyat"])[0]
```

VERİMLİ

```
def en_ucuz_urun(urunler):
    return min(urunler, key=lambda u: u["fiyat"])
```

ÖLÇÜM süre 3,5 ms → 827 µs · bellek 468,9 KB → 208 B · aynı sonuç ✓

Motorun düzeltmesi de aynı sonucu verdi (4,38×).

Kaynak: Ruff FURB192 sorted-min-max · `docs.astral.sh/ruff/rules/sorted-min-max/`

YK013 yüksek Öneri verir **64,3× hızlı**

Döngüde listede index() ya da count()

index() ve count() listeyi baştan sona tarar; döngüde iş karesel büyür. Konumlar için sözlük, sayım için collections.Counter kullanın.

NEDEN ENERJİ HARCAR?

list.index ve list.count her çağrıda listeyi tarar; döngüde n × m adım eder. Bir kez hazırlanan sözlük ya da Counter ile aramalar sabit süredir.

NE ZAMAN SORUN DEĞİL?

Liste küçükse ya da çağrı bir kez yapılıyorsa sorun değildir.

YAPILAN İŞ **O(n·m) → O(n + m)**

İSRAFLI

```
def siralama_puani(ogrenciler, sinif_listesi):
    sira = list(sinif_listesi)
    return [sira.index(o) for o in ogrenciler]
```

VERİMLİ

```
def siralama_puani(ogrenciler, sinif_listesi):
    sira = {}
    for i, ad in enumerate(sinif_listesi):
        sira.setdefault(ad, i)
    return [sira[o] for o in ogrenciler]
```

ÖLÇÜM süre 14,1 ms → 220 µs · bellek 57,0 KB → 219,8 KB · aynı sonuç ✓

Kaynak: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK026 yüksek Öneri verir **292× hızlı**

Döngüde sözlük değerlerinde arama

x in d.values() bütün değerleri tek tek tarar. Değerlerden bir küme (set) oluşturun ya da ters sözlük tutun.

NEDEN ENERJİ HARCAR?

values() görünümü bir karma tablosu değildir; 'in' bütün değerleri tek tek karşılaştırır.

NE ZAMAN SORUN DEĞİL?

Sözlük küçükse ya da arama bir kez yapılıyorsa sorun değildir.

YAPILAN İŞ **O(n·m) → O(n + m)**

İSRAFLI

```
def kayitli_eposta(adaylar, kullanicilar):
    return [e for e in adaylar if e in kullanicilar.values()]
```

VERİMLİ

```
def kayitli_eposta(adaylar, kullanicilar):
    epostalar = set(kullanicilar.values())
    return [e for e in adaylar if e in epostalar]
```

ÖLÇÜM süre 16,4 ms → 56 µs · bellek 4,2 KB → 160,3 KB · aynı sonuç ✓

Kaynak: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK028 orta Öneri verir **4,70× hızlı**

Döngüde değişmeyen metni yeniden işleme

Döngü boyunca değişmeyen bir metin her adımda yeniden bölünüyor ya da dönüştürülüyor. İşlemi döngüden önce bir kez yapın.

NEDEN ENERJİ HARCAR?

Değişmeyen bir metni her adımda bölmek ya da küçük harfe çevirmek aynı sonucu tekrar tekrar üretir.

NE ZAMAN SORUN DEĞİL?

Metin döngüde değişiyorsa işlem her adımda gereklidir.

YAPILAN İŞ **n iş → 1 iş**

İSRAFLI

```
def yasakli_kelime_var_mi(mesajlar, yasakli_metin):
    sonuc = []
    for m in mesajlar:
        yasaklar = yasakli_metin.lower().split(",")
        sonuc.append(any(y in m for y in yasaklar))
    return sonuc
```

VERİMLİ

```
def yasakli_kelime_var_mi(mesajlar, yasakli_metin):
    yasaklar = yasakli_metin.lower().split(",")
    return [any(y in m for y in yasaklar) for m in mesajlar]
```

ÖLÇÜM süre 4,9 ms → 1,0 ms · bellek 30,2 KB → 28,1 KB · aynı sonuç ✓

Kaynak: Python wiki: PerformanceTips · wiki.python.org/moin/PythonSpeed/PerformanceTips

---

YK041 orta Öneri verir **20,1× hızlı**

İlk eşleşme için bütün listeyi üretme

[...][0] bütün listeyi üretip yalnızca ilk elemanı kullanır. next((x for x in ... if ...), None) ilk eşleşmede durur.

NEDEN ENERJİ HARCAR?
Liste kurgusu bütün elemanları üretir; yalnızca ilki gerekiyorsa next() ilk eşleşmede durur.

NE ZAMAN SORUN DEĞİL?
Liste sonra başka işler için de kullanılıyorsa fark yoktur.

YAPILAN İŞ **O(n) → ilk eşleşmeye kadar**

İSRAFLI

```
def ilk_buyuk_siparis(siparisler):
    return [s for s in siparisler if s["tutar"] > 1000][0]
```

VERİMLİ

```
def ilk_buyuk_siparis(siparisler):
    return next((s for s in siparisler if s["tutar"] > 1000),
  None)
```

ÖLÇÜM süre 398 µs → 20 µs · bellek 80 B → 408 B · aynı sonuç ✓

Kaynak: Python belgeleri: next() ve üreteçler · docs.python.org/3/library/functions.html#next

---

YK042 orta Öneri verir **2.438× hızlı**

İlk eleman için bütün listeyi oluşturma

list(x)[0] bütün elemanları kopyalar. next(iter(x)) yalnızca ilk elemanı alır.

NEDEN ENERJİ HARCAR?
list(x) bütün elemanları kopyalar; next(iter(x)) yalnızca ilkini alır.

NE ZAMAN SORUN DEĞİL?
Liste sonra başka işler için de kullanılıyorsa fark yoktur.

YAPILAN İŞ **O(n) → O(1)**

İSRAFLI

```
def ilk_anahtar(sozluk):
    return list(sozluk.keys())[0]
```

VERİMLİ

```
def ilk_anahtar(sozluk):
    return next(iter(sozluk))
```

ÖLÇÜM süre 230 µs → 0,1 µs · bellek 390,7 KB → 72 B · aynı sonuç ✓

Kaynak: Python belgeleri: next(iter(...)) · docs.python.org/3/library/functions.html#next

---

YK043 bilgi Öneri verir **13,7× hızlı**

İlk k eleman için tam sıralama

sorted(x)[:k] bütün listeyi sıralar. Küçük k için heapq.nsmallest(k, x) ya da heapq.nlargest(k, x) daha az iş yapar.

NEDEN ENERJİ HARCAR?
Tam sıralama O(n log n) iştir; heapq.nsmallest küçük k için O(n log k) ile yetinir ve tam kopya oluşturmaz.

NE ZAMAN SORUN DEĞİL?
k, n'ye yakınsa tam sıralama daha hızlı olabilir.

YAPILAN İŞ **O(n log n) → O(n log k)**

İSRAFLI

```
def en_iyi_uc(puanlar):
    return sorted(puanlar, reverse=True)[:3]
```

VERİMLİ

```
import heapq


def en_iyi_uc(puanlar):
    return heapq.nlargest(3, puanlar)
```

ÖLÇÜM süre 7,4 ms → 542 µs · bellek 585,9 KB → 316 B · aynı sonuç ✓

Kaynak: Python belgeleri: heapq.nsmallest · docs.python.org/3/library/heapq.html#heapq.nsmallest

---

YK044 yüksek Öneri verir 439× hızlı

Döngüde arama için koleksiyon oluşturma

x in set(...) ya da list(...) döngünün her adımında koleksiyonu baştan kurar. Kümeyi döngüden önce bir kez oluşturun.

NEDEN ENERJİ HARCAR?
set(...) her adımda bütün kaynağı yeniden karma tablosuna doldurur; kazanılması beklenen hız kaybolur, üstüne kurulum maliyeti eklenir.

NE ZAMAN SORUN DEĞİL?
Kaynak her adımda değişiyorsa yeniden kurmak gerekir.

YAPILAN İŞ **O(n·m) → O(n + m)**

İSRAFLI

```
def kayitli_olanlar(gelenler, kayitlar):
    return [g for g in gelenler if g in set(kayitlar)]
```

VERİMLİ

```
def kayitli_olanlar(gelenler, kayitlar):
    kayitli = set(kayitlar)
    return [g for g in gelenler if g in kayitli]
```

ÖLÇÜM süre 16,0 ms → 36 µs · bellek 164,4 KB → 160,3 KB · aynı sonuç ✓

Kaynak: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK045 orta Öneri verir 462× hızlı

Döngüde listeden remove()

remove() listeyi tarar ve arkadaki elemanları kaydırır; döngüde iş karesel büyür. Kalacak elemanlarla yeni liste kurun ya da küme kullanın.

NEDEN ENERJİ HARCAR?
remove önce elemanı arar, sonra arkadakileri kaydırır; döngüde karesel iş eder.

NE ZAMAN SORUN DEĞİL?
Birkaç eleman çıkarılıyorsa sorun değildir.

YAPILAN İŞ **O(n·m) → O(n + m)**

İSRAFLI

```
def gecersizleri_cikar(kayitlar, gecersizler):
    kalan = list(kayitlar)
    for g in gecersizler:
        kalan.remove(g)
    return kalan
```

VERİMLİ

```
def gecersizleri_cikar(kayitlar, gecersizler):
    cikarilacak = set(gecersizler)
    return [k for k in kayitlar if k not in cikarilacak]
```

ÖLÇÜM süre 170,2 ms → 368 µs · bellek 156,4 KB → 278,4 KB · aynı sonuç ✓

Kayıtlar benzersiz; yinelenen kayıtlarda remove yalnızca ilkini siler, iyi sürüm hepsini.

Kaynak: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK047 orta Öneri verir 8,20× hızlı

Döngüde büyüyen listeyi her adımda birleştirme

Her adımda büyüyen listeyi join() ile baştan birleştirmek karesel iştir. Birleştirmeyi döngüden sonra bir kez yapın.

NEDEN ENERJİ HARCAR?
join her çağrıda bütün listeyi baştan birleştirir; liste büyüdükçe her adımın maliyeti artar ve toplam iş karesel olur.

NE ZAMAN SORUN DEĞİL?
Ara sonuçların her biri gerçekten gerekiyorsa son metni adım adım büyütmek daha iyidir.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
def ilerleme_mesajlari(adimlar):
    parcalar = []
    mesajlar = []
    for a in adimlar:
        parcalar.append(a)
        mesajlar.append(" > ".join(parcalar))
    return mesajlar
```

VERİMLİ

```
from itertools import accumulate


def ilerleme_mesajlari(adimlar):
    return list(accumulate(adimlar, lambda once, a: once + " >
  " + a))
```

ÖLÇÜM süre 3,9 ms → 482 µs · bellek 6,6 MB → 6,6 MB · aynı sonuç ✓

Kaynak: Python wiki: PerformanceTips (str.join) · wiki.python.org/moin/PythonSpeed/PerformanceTips

---

YK048 orta Öneri verir **114× hızlı**

Döngüde derin kopya

copy.deepcopy nesnenin bütün alt yapısını dolaşır ve kopyalar; döngüde çok pahalıdır. Yalnızca değişen kısmı kopyalayın ya da değişmez yapılar kullanın.

NEDEN ENERJİ HARCAR?

deepcopy bütün nesne grafiğini dolaşır, her alt nesneyi kopyalar ve döngüleri izlemek için kayıt tutar; Python'daki en pahalı işlemlerden biridir.

NE ZAMAN SORUN DEĞİL?

Gerçekten bağımsız ve değiştirilebilir kopyalar gerekiyorsa kaçınılmaz olabilir; çoğu zaman yalnızca değişen alanı kopyalamak yeter.

YAPILAN İŞ `tam grafik kopyası → yalnızca değişen kısım`

İSRAFLI

```
import copy


def raporlar(sablon, ogrenciler):
    sonuc = []
    for o in ogrenciler:
        r = copy.deepcopy(sablon)
        r["ad"] = o
        sonuc.append(r)
    return sonuc
```

VERİMLİ

```
def raporlar(sablon, ogrenciler):
    return [{**sablon, "ad": o} for o in ogrenciler]
```

ÖLÇÜM süre 12,4 ms → 109 µs · bellek 1,7 MB → 184,0 KB · aynı sonuç ✓

İyi sürüm sığ kopya yapar: iç içe 'dersler' listesi raporlar arasında paylaşılır. Raporlar sonradan tek tek değiştirilecekse yalnızca değişen kısmı kopyalayın.

Kaynak: Python belgeleri: copy · `docs.python.org/3/library/copy.html`

YK049 bilgi Öneri verir **1,15× yavaş**

Döngü içinde try/except

Python 3.11 öncesinde her döngü adımında try bloğu kurmak ek iş getirir. Mümkünse try/except'i döngünün dışına alın.

NEDEN ENERJİ HARCAR?

Python 3.11 öncesinde her try bloğuna girişte istisna işleme için kurulum yapılırdı. 3.11'in sıfır maliyetli istisnalarıyla bu maliyet büyük ölçüde kalktı.

NE ZAMAN SORUN DEĞİL?

Python 3.11 ve sonrasında hata oluşmadıkça maliyet yok denecek kadar azdır.

YAPILAN İŞ `her adımda kurulum → yok (3.11+)`

İSRAFLI

```
def sayilara_cevir(metinler):
    sonuc = []
    for m in metinler:
        try:
            sonuc.append(int(m))
        except ValueError:
            sonuc.append(0)
    return sonuc
```

VERİMLİ

```
def _sayi(metin):
    try:
        return int(metin)
    except ValueError:
        return 0


def sayilara_cevir(metinler):
    return [_sayi(m) for m in metinler]
```

ÖLÇÜM süre 2,3 ms → 2,6 ms · bellek 655,1 KB → 655,0 KB · aynı sonuç ✓

Python 3.11'den beri try bloğuna girmenin maliyeti yok; bu ölçümün yapıldığı sürümde fark beklenmez.

Kaynak: Ruff PERF203 try-except-in-loop · `docs.astral.sh/ruff/rules/try-except-in-loop/`

YK059 bilgi Öneri verir

2,71× hızlı

Döngü içinde import

import her adımda modül tablosunda arama yapar. İçe aktarmayı dosyanın başına alın.

NEDEN ENERJİ HARCAR?

import her seferinde sys.modules'te arama ve isim bağlama yapar; modül önbellekte olsa da adım başına ek iş getirir.

NE ZAMAN SORUN DEĞİL?

Ağır, isteğe bağlı modülleri geç yüklemek için fonksiyon içinde import mantıklıdır, ama döngünün dışında.

YAPILAN İŞ **n arama → 1**

İSRAFLI

```
def tarihleri_isle(metinler):
    sonuc = []
    for m in metinler:
        from datetime import datetime
        sonuc.append(datetime.fromisoformat(m))
    return sonuc
```

VERİMLİ

```
from datetime import datetime


def tarihleri_isle(metinler):
    return [datetime.fromisoformat(m) for m in metinler]
```

ÖLÇÜM süre 7,1 ms → 2,6 ms · bellek 950,3 KB → 950,3 KB · aynı sonuç ✓

Kaynak: Python wiki: PerformanceTips (Import Statement Overhead) · wiki.python.org/moin/PythonSpeed/PerformanceTips#Import_Statement_Overhead

---

YK068 orta Öneri verir

2,83× hızlı

Döngüde değişmeyen matematik hesabı

Bu math çağrısının sonucu döngü boyunca aynı. Döngüden önce bir kez hesaplayıp sonucu kullanın.

NEDEN ENERJİ HARCAR?

Değişmeyen bir matematik hesabı her adımda tekrar yapılır; sonuç bir kez hesaplanıp saklanabilir.

NE ZAMAN SORUN DEĞİL?

Hesap çok ucuzsa (math.pi gibi sabit) fark yoktur.

YAPILAN İŞ **n hesap → 1**

İSRAFLI

```
import math


def ucgen_yukseklikleri(kenarlar):
    return [k * math.sqrt(3) / 2 for k in kenarlar]
```

VERİMLİ

```
import math


def ucgen_yukseklikleri(kenarlar):
    oran = math.sqrt(3) / 2
    return [k * oran for k in kenarlar]
```

ÖLÇÜM süre 3,0 ms → 1,1 ms · bellek 1,6 MB → 1,6 MB · aynı sonuç ✓

Kaynak: Python wiki: PerformanceTips · wiki.python.org/moin/PythonSpeed/PerformanceTips

---

03 Veri yapıları ve Python dili

31 kural

YK009 orta Öneri verir 71,8× hızlı

Listenin başından ekleme ya da çıkarma

pop(0) ve insert(0, ...) listenin bütün elemanlarını kaydırır. collections.deque kullanın: popleft ve appendleft sabit sürede çalışır.

NEDEN ENERJİ HARCAR?
Liste bitişik bir bellek dizisidir; baştan eleman çıkarmak ya da eklemek arkadaki bütün elemanları kaydırır. deque iki uçtan da sabit sürede çalışır.

NE ZAMAN SORUN DEĞİL?
Liste çok kısaysa fark ölçülemeyecek kadar küçüktür.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
def sirayla_isle(isler):
    biten = []
    while isler:
        biten.append(isler.pop(0))
    return biten
```

VERİMLİ

```
from collections import deque


def sirayla_isle(isler):
    kuyruk = deque(isler)
    isler.clear()
    biten = []
    while kuyruk:
        biten.append(kuyruk.popleft())
    return biten
```

ÖLÇÜM süre 91,2 ms → 1,3 ms · bellek 300,8 KB → 270,8 KB · aynı sonuç ✓

Kaynak: Python belgeleri: collections.deque · `docs.python.org/3/library/collections.html#collections.deque`

YK016 orta Öneri verir 13,9× hızlı

any() ya da all() içinde liste

Köşeli parantezli liste önce bütün elemanları hesaplar; any() ve all() ilk sonuçta duramaz. Köşeli parantezleri kaldırıp üreteç kullanın.

NEDEN ENERJİ HARCAR?
any([...]) önce bütün listeyi hesaplar; ilk True'da durma fırsatı kaybolur. Üreteçle any ilk eşleşmede biter.

NE ZAMAN SORUN DEĞİL?
Her elemanın yan etkisi için hesaplanması gerekiyorsa (bilerek) liste kullanılabilir.

YAPILAN İŞ **n → ilk eşleşmeye kadar**

İSRAFLI

```
def gecersiz_var_mi(kayitlar):
    return any([dogrula(k) is False for k in kayitlar])
```

VERİMLİ

```
def gecersiz_var_mi(kayitlar):
    return any(dogrula(k) is False for k in kayitlar)
```

ÖLÇÜM süre 624 µs → 45 µs · bellek 169,0 KB → 416 B · aynı sonuç ✓

Geçersiz kayıt listenin başlarında (1.000. sırada). Sonda olsaydı iki sürüm aynı işi yapardı.

Kaynak: Ruff C419 unnecessary-comprehension-in-call · `docs.astral.sh/ruff/rules/unnecessary-comprehension-in-call/`

YK017 orta Öneri verir 118× hızlı

Döngüde listeyi yeniden oluşturma

L = L + [x] her adımda bütün listeyi kopyalar; iş karesel büyür. L.append(x) ya da L.extend(...) kullanın.

NEDEN ENERJİ HARCAR?
+ işleci her seferinde iki listenin tamamını yeni bir listeye kopyalar; append sona yerinde ekler.

NE ZAMAN SORUN DEĞİL?
Listenin eski hâline başka bir değişken bağlıysa ve o hâl korunmalıysa kopya bilinçli olabilir.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
def ciftleri_topla(sayilar):
    sonuc = []
    for s in sayilar:
        if s % 2 == 0:
            sonuc = sonuc + [s]
    return sonuc
```

VERİMLİ

```
def ciftleri_topla(sayilar):
    return [s for s in sayilar if s % 2 == 0]
```

ÖLÇÜM süre 37,7 ms → 320 µs · bellek 78,2 KB → 40,9 KB · aynı sonuç ✓

Kaynak: Python wiki: TimeComplexity · `wiki.python.org/moin/TimeComplexity`

YK024 orta · Öneri verir — **111× hızlı**

JSON ile kopyalama

json.loads(json.dumps(x)) veriyi metne çevirip geri ayrıştırır; demetleri listeye, sayı anahtarlarını metne çevirir. Yalnızca değişecek kısmı kopyalayın: {**x, 'alan': yeni}. Tam bağımsız kopya gerekiyorsa copy.deepcopy türleri korur.

NEDEN ENERJİ HARCAR?

JSON gidiş-dönüşü her değeri metne çevirip yeniden ayrıştırır; demetleri listeye, sayı anahtarlarını metne çevirip veriyi sessizce bozar. Çoğu zaman bütün yapıyı kopyalamak da gerekmez: yalnızca değişecek alanı içeren yeni bir sözlük, geri kalanı paylaşarak aynı sonucu verir.

NE ZAMAN SORUN DEĞİL?

Kopya sonradan iç içe alanlarında değiştirilecekse sığ kopya yetmez; o zaman copy.deepcopy kullanın. deepcopy türleri korur ama JSON'dan da yavaştır, yalnızca gerçekten gerekiyorsa kullanın.

YAPILAN İŞ **tam kopya → yalnızca değişen alan**

İSRAFLI

```
import json


def kullanici_ayari(varsayilan, tema):
    ayar = json.loads(json.dumps(varsayilan))
    ayar["tema"] = tema
    return ayar
```

VERİMLİ

```
def kullanici_ayari(varsayilan, tema):
    return {**varsayilan, "tema": tema}
```

ÖLÇÜM süre 16 µs → 0,1 µs · bellek 4,7 KB → 120 B · aynı sonuç ✓

İyi sürüm yalnızca değişen alanı kopyalar; iç içe yapılar varsayılan ayarla paylaşılır. Karşılaştırmada varsayılan ayarın değişmediği de denetlendi.

Kaynak: Python belgeleri: copy · json (dönüşüm tablosu) · `docs.python.org/3/library/json.html#py-to-json-table`

YK025 orta · Öneri verir — **178× hızlı**

sum() ile liste birleştirme

sum(listeler, []) her adımda yeni bir liste oluşturur; iş karesel büyür. itertools.chain.from_iterable kullanın.

NEDEN ENERJİ HARCAR?

sum her adımda biriken listeyle yeni elemanı toplayıp yepyeni bir liste oluşturur; toplam kopya miktarı karesel büyür. chain elemanları kopyalamadan sırayla verir.

NE ZAMAN SORUN DEĞİL?

Birkaç kısa liste birleştiriliyorsa fark önemsizdir.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
def hepsini_birlestir(gruplar):
    return sum(gruplar, [])
```

VERİMLİ

```
from itertools import chain


def hepsini_birlestir(gruplar):
    return list(chain.from_iterable(gruplar))
```

ÖLÇÜM süre 13,9 ms → 78 µs · bellek 156,2 KB → 83,3 KB · aynı sonuç ✓

Kaynak: Python belgeleri: sum() (itertools.chain önerisi) · `docs.python.org/3/library/functions.html#sum`

YK029 bilgi · Motor düzeltir — **1,20× hızlı**

Gereksiz liste kopyası

Döngüden önce list() ile kopya oluşturmak bellek ve zaman harcar. range, map, zip gibi nesneler doğrudan gezilebilir.

NEDEN ENERJİ HARCAR?

list() bütün elemanları önce bellekte kopyalar; range, map ve zip tembeldir ve doğrudan gezilebilir.

NE ZAMAN SORUN DEĞİL?

Gezinme sırasında kaynak değiştiriliyorsa (sözlükten silme) kopya bilinçli olarak gerekir.

YAPILAN İŞ **O(n) bellek → O(1)**

İSRAFLI

```
def kareler(n):
    toplam = 0
    for i in list(range(n)):
        toplam += i * i
    return toplam
```

VERİMLİ

```
def kareler(n):
    toplam = 0
    for i in range(n):
        toplam += i * i
    return toplam
```

ÖLÇÜM süre 9,8 ms → 8,2 ms · bellek 7,6 MB → 176 B · aynı sonuç ✓

Süre benzer; asıl kazanç bellektedir: range sayıları tek tek üretir, list() hepsini birden tutar.

Motorun düzeltmesi de aynı sonucu verdi (1,24×).

Kaynak: Ruff PERF101 unnecessary-list-cast · `docs.astral.sh/ruff/rules/unnecessary-list-cast/`

YK046 orta Öneri verir **139× hızlı**

Döngüde del L[0]

Listenin ilk elemanını silmek bütün elemanları kaydırır. collections.deque ve popleft() kullanın.

NEDEN ENERJİ HARCAR?
Listenin başından silmek arkadaki bütün elemanları bir öne kaydırır; deque.popleft sabit süredir.

NE ZAMAN SORUN DEĞİL?
Liste çok kısaysa fark küçüktür.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
def kuyrugu_isle(isler):
    sonuc = []
    while isler:
        sonuc.append(isler[0] * 2)
        del isler[0]
    return sonuc
```

VERİMLİ

```
from collections import deque


def kuyrugu_isle(isler):
    kuyruk = deque(isler)
    isler.clear()
    sonuc = []
    while kuyruk:
        sonuc.append(kuyruk.popleft() * 2)
    return sonuc
```

ÖLÇÜM süre 220,2 ms → 1,6 ms · bellek 1,1 MB → 1,2 MB · aynı sonuç ✓

Kaynak: Python belgeleri: collections.deque · docs.python.org/3/library/collections.html#collections.deque

---

YK050 bilgi Öneri verir **1,15× hızlı**

Elle liste kurgusu

Boş listeye döngüde append() yapmak, aynı işi yapan liste kurgusundan (list comprehension) yavaştır.

NEDEN ENERJİ HARCAR?
Döngüde her append bir öznitelik araması ve metot çağrısıdır; liste kurgusu özel bir bayt kodu (LIST_APPEND) kullanır.

NE ZAMAN SORUN DEĞİL?
Döngü gövdesi karmaşıksa okunabilirlik için açık döngü tercih edilebilir; fark genellikle %10–30'dur.

YAPILAN İŞ **n metot çağrısı → 0**

İSRAFLI

```
def kareler(sayilar):
    sonuc = []
    for s in sayilar:
        sonuc.append(s * s)
    return sonuc
```

VERİMLİ

```
def kareler(sayilar):
    return [s * s for s in sayilar]
```

ÖLÇÜM süre 520 µs → 452 µs · bellek 793,4 KB → 793,4 KB · aynı sonuç ✓

Kaynak: Ruff PERF401 manual-list-comprehension · docs.astral.sh/ruff/rules/manual-list-comprehension/

YK051 bilgi Öneri verir **4,29× hızlı**

Elle liste kopyalama

Elemanları döngüyle tek tek eklemek yerine list(x) ya da L.extend(x) kullanın.

NEDEN ENERJİ HARCAR?
Eleman eleman kopyalamak Python düzeyinde döner; list(x) ve extend kopyayı C ile tek seferde yapar.

NE ZAMAN SORUN DEĞİL?
Kopyalarken eleman dönüştürülüyorsa liste kurgusu kullanın.

YAPILAN İŞ **Python döngüsü → C kopyası**

İSRAFLI

```
def kopyala(kaynak):
    hedef = []
    for x in kaynak:
        hedef.append(x)
    return hedef
```

VERİMLİ

```
def kopyala(kaynak):
    return list(kaynak)
```

ÖLÇÜM süre 262 µs → 61 µs · bellek 169,0 KB → 156,3 KB · aynı sonuç ✓

Kaynak: Ruff PERF402 manual-list-copy · docs.astral.sh/ruff/rules/manual-list-copy/

YK052 bilgi Öneri verir

fark yok

Elle sözlük kurgusu

Boş sözlüğü döngüde doldurmak, sözlük kurgusundan (dict comprehension) yavaştır.

NEDEN ENERJİ HARCAR?

Döngüde her atama ayrı bir Python adımıdır; sözlük kurgusu aynı işi özel bayt koduyla yapar.

NE ZAMAN SORUN DEĞİL?

Döngü gövdesi karmaşıksa açık döngü daha okunaklı olabilir.

YAPILAN İŞ **n adım → kurgu**

İSRAFLI

```
def numara_sozlugu(ogrenciler):
    sozluk = {}
    for o in ogrenciler:
        sozluk[o["no"]] = o["ad"]
    return sozluk
```

VERİMLİ

```
def numara_sozlugu(ogrenciler):
    return {o["no"]: o["ad"] for o in ogrenciler}
```

ÖLÇÜM süre 797 µs → 787 µs · bellek 864,1 KB → 864,1 KB · aynı sonuç ✓

Kaynak: Ruff PERF403 manual-dict-comprehension · docs.astral.sh/ruff/rules/manual-dict-comprehension/

YK053 bilgi Öneri verir

1,76× hızlı

Döngüde kümeye tek tek ekleme

Döngüde s.add(x) yerine s.update(kaynak) tek çağrıda ekler.

NEDEN ENERJİ HARCAR?

Her add bir metot çağrısıdır; update bütün elemanları C döngüsünde ekler.

NE ZAMAN SORUN DEĞİL?

Eklerken eleman dönüştürülüyorsa küme kurgusu kullanın.

YAPILAN İŞ **n çağrı → 1**

İSRAFLI

```
def etiketleri_topla(yazilar):
    etiketler = set()
    for e in yazilar:
        etiketler.add(e)
    return etiketler
```

VERİMLİ

```
def etiketleri_topla(yazilar):
    etiketler = set()
    etiketler.update(yazilar)
    return etiketler
```

ÖLÇÜM süre 510 µs → 290 µs · bellek 40,3 KB → 40,3 KB · aynı sonuç ✓

Kaynak: Ruff FURB142 for-loop-set-mutations · docs.astral.sh/ruff/rules/for-loop-set-mutations/

YK054 bilgi Öneri verir

4,90× hızlı

items() ile kullanılmayan anahtar ya da değer

items() hem anahtarı hem değeri üretir ama biri kullanılmıyor. Yalnızca keys() ya da values() gezin.

NEDEN ENERJİ HARCAR?

items() her adımda bir anahtar-değer demeti oluşturur; biri kullanılmıyorsa keys() ya da values() daha az iş yapar.

NE ZAMAN SORUN DEĞİL?

Fark küçüktür; büyük sözlüklerde anlamlıdır.

YAPILAN İŞ **demet → tek değer**

İSRAFLI

```
def toplam_puan(puanlar):
    toplam = 0
    for ad, puan in puanlar.items():
        toplam += puan
    return toplam
```

VERİMLİ

```
def toplam_puan(puanlar):
    return sum(puanlar.values())
```

ÖLÇÜM süre 488 µs → 100 µs · bellek 136 B → 112 B · aynı sonuç ✓

Kaynak: Ruff PERF102 incorrect-dict-iterator · docs.astral.sh/ruff/rules/incorrect-dict-iterator/

YK055 bilgi Öneri verir 1,32× hızlı

Anahtarlarla gezip değeri yeniden arama

for k in d: ... d[k] her adımda sözlükte ikinci bir arama yapar. for k, v in d.items() kullanın.

NEDEN ENERJİ HARCAR?

d[k] her adımda karma hesaplayıp sözlükte yeniden arar; items() değeri anahtarla birlikte verir.

NE ZAMAN SORUN DEĞİL?

Sözlük döngüde değiştiriliyorsa items() yerine anahtar listesi gerekebilir.

YAPILAN İŞ **2 arama → 1**

İSRAFLI

```
def rapor(stok):
    satirlar = []
    for urun in stok:
        satirlar.append(f"{urun}: {stok[urun]}")
    return satirlar
```

VERİMLİ

```
def rapor(stok):
    return [f"{urun}: {adet}" for urun, adet in stok.items()]
```

ÖLÇÜM süre 2,7 ms → 2,0 ms · bellek 1,5 MB → 1,5 MB · aynı sonuç ✓

Kaynak: Pylint C0206 consider-using-dict-items · pylint.readthedocs.io/en/stable/user_guide/messages/convention/consider-using-dict-items.html

---

YK056 bilgi Öneri verir 1,20× hızlı

range(len()) ile indeksleme

for i in range(len(x)): x[i] her adımda indeksleme yapar. for i, eleman in enumerate(x) kullanın.

NEDEN ENERJİ HARCAR?

x[i] her adımda indeksleme ve sınır kontrolü yapar; enumerate elemanı doğrudan verir.

NE ZAMAN SORUN DEĞİL?

İndeksle başka bir diziye de erişiliyorsa zip ya da enumerate birlikte kullanılabilir.

YAPILAN İŞ **indeksleme → doğrudan eleman**

İSRAFLI

```
def numarali(ogrenciler):
    sonuc = []
    for i in range(len(ogrenciler)):
        sonuc.append((i + 1, ogrenciler[i]))
    return sonuc
```

VERİMLİ

```
def numarali(ogrenciler):
    return list(enumerate(ogrenciler, 1))
```

ÖLÇÜM süre 1,8 ms → 1,5 ms · bellek 1,7 MB → 1,7 MB · aynı sonuç ✓

Kaynak: Pylint C0200 consider-using-enumerate · pylint.readthedocs.io/en/stable/user_guide/messages/convention/consider-using-enumerate.html

---

YK057 bilgi Öneri verir 1,98× hızlı

Sayım için elle sözlük

if k in d: d[k] += 1 else: d[k] = 1 her adımda iki arama yapar. collections.Counter tek geçişte sayar.

NEDEN ENERJİ HARCAR?

Her adımda önce üyelik kontrolü, sonra okuma ve yazma yapılır; Counter sayımı C ile tek geçişte yapar.

NE ZAMAN SORUN DEĞİL?

Sayım dışında karmaşık bir birleştirme yapılıyorsa elle sözlük gerekebilir.

YAPILAN İŞ **3 işlem → 1**

İSRAFLI

```
def kelime_say(kelimeler):
    sayac = {}
    for k in kelimeler:
        if k in sayac:
            sayac[k] += 1
        else:
            sayac[k] = 1
    return sayac
```

VERİMLİ

```
from collections import Counter


def kelime_say(kelimeler):
    return Counter(kelimeler)
```

ÖLÇÜM süre 2,5 ms → 1,2 ms · bellek 38,1 KB → 38,2 KB · aynı sonuç ✓

Kaynak: Python belgeleri: collections.Counter · docs.python.org/3/library/collections.html#collections.Counter

---

YK058 bilgi Öneri verir **1,41× hızlı**

Gruplama için elle sözlük

if k not in d: d[k] = [] kalıbı her adımda ek arama yapar. collections.defaultdict(list) kullanın.

NEDEN ENERJİ HARCAR?
Her adımda ek bir üyelik kontrolü yapılır; defaultdict eksik anahtarı ilk erişimde kendisi oluşturur.

NE ZAMAN SORUN DEĞİL?
Eksik anahtarın kendiliğinden oluşması istenmiyorsa (okuma sırasında) normal sözlük daha güvenlidir.

YAPILAN İŞ **2 arama → 1**

İSRAFLI

```
def sehre_gore(musteriler):
    gruplar = {}
    for m in musteriler:
        if m["sehir"] not in gruplar:
            gruplar[m["sehir"]] = []
        gruplar[m["sehir"]].append(m)
    return gruplar
```

VERİMLİ

```
from collections import defaultdict


def sehre_gore(musteriler):
    gruplar = defaultdict(list)
    for m in musteriler:
        gruplar[m["sehir"]].append(m)
    return dict(gruplar)
```

ÖLÇÜM süre 1,2 ms → 820 µs · bellek 175,4 KB → 176,8 KB · aynı sonuç ✓

Kaynak: Python belgeleri: collections.defaultdict · docs.python.org/3/library/collections.html#collections.defaultdict

---

YK060 bilgi Öneri verir **1,50× hızlı**

Döngüde boş koleksiyon için dict()/list() çağrısı

dict() ve list() isim araması ve fonksiyon çağrısı yapar; {} ve [] doğrudan oluşturur.

NEDEN ENERJİ HARCAR?
dict() global isim araması ve fonksiyon çağrısıdır; {} doğrudan bayt koduyla oluşturulur.

NE ZAMAN SORUN DEĞİL?
Anahtar kelime argümanlarıyla dict(a=1) okunaklılık için tercih edilebilir.

YAPILAN İŞ **çağrı → literal**

İSRAFLI

```
def bos_kayitlar(n):
    kayitlar = []
    for i in range(n):
        kayitlar.append(dict())
    return kayitlar
```

VERİMLİ

```
def bos_kayitlar(n):
    return [{} for _ in range(n)]
```

ÖLÇÜM süre 1,4 ms → 919 µs · bellek 1,4 MB → 1,4 MB · aynı sonuç ✓

Kaynak: Ruff C408 unnecessary-collection-call · docs.astral.sh/ruff/rules/unnecessary-collection-call/

YK061 bilgi Öneri verir **2,12× hızlı**

map() ya da filter() ile lambda

map(lambda ...) her eleman için ayrı bir fonksiyon çağrısı yapar. Liste kurgusu ya da üreteç daha hızlıdır.

NEDEN ENERJİ HARCAR?
lambda her eleman için Python fonksiyon çağrısı (çerçeve kurma) gerektirir; kurgu ifadeyi satır içinde çalıştırır.

NE ZAMAN SORUN DEĞİL?
Hazır bir yerleşik fonksiyonla map hızlıdır (map(str, x)); sorun lambda'dadır.

YAPILAN İŞ **n fonksiyon çağrısı → 0**

İSRAFLI

```
def kdvli(fiyatlar):
    return list(map(lambda f: f * 1.2, fiyatlar))
```

VERİMLİ

```
def kdvli(fiyatlar):
    return [f * 1.2 for f in fiyatlar]
```

ÖLÇÜM süre 1,0 ms → 490 µs · bellek 637,9 KB → 637,7 KB · aynı sonuç ✓

Kaynak: Ruff C417 unnecessary-map · docs.astral.sh/ruff/rules/unnecessary-map/

YK062 bilgi Motor düzeltir — fark yok

Gereksiz çift dönüşüm

list(sorted(x)), sorted(list(x)), set(list(x)) gibi çağrılar veriyi gereksiz yere bir kez daha kopyalar.

NEDEN ENERJİ HARCAR?
İç çağrı zaten yeni bir liste ya da küme üretir; dıştaki çağrı aynı veriyi bir kez daha kopyalar.

NE ZAMAN SORUN DEĞİL?
tuple(sorted(x)) gibi tür değişimi gerçekten gerekiyorsa çift çağrı doğrudur.

YAPILAN İŞ **2 kopya → 1**

İSRAFLI

```
def benzersiz_sirali(etiketler):
    return list(sorted(set(etiketler)))
```

VERİMLİ

```
def benzersiz_sirali(etiketler):
    return sorted(set(etiketler))
```

ÖLÇÜM süre 858 µs → 805 µs · bellek 163,4 KB → 163,4 KB · aynı sonuç ✓

Fark ölçülemeyecek kadar küçük: fazladan kopya C ile yapılır ve sıralamanın yanında önemsizdir. Kazanç sadeliktedir.

Motorun düzeltmesi de aynı sonucu verdi (1,26×).

Kaynak: Ruff C414 unnecessary-double-cast-or-process · docs.astral.sh/ruff/rules/unnecessary-double-cast-or-process/

YK063 bilgi Öneri verir — fark yok

reversed(sorted(...)) ya da sorted(...)[::-1]

Sıralayıp sonra ters çevirmek fazladan bir geçiştir. sorted(x, reverse=True) kullanın.

NEDEN ENERJİ HARCAR?
Sıralama sonrası ters çevirmek ek bir geçiş ve kopyadır; reverse=True aynı işi sıralama içinde yapar.

NE ZAMAN SORUN DEĞİL?
Eşit elemanların sırası önemliyse dikkat: reversed(sorted()) eşitleri ters sıraya koyar, reverse=True korur.

YAPILAN İŞ **2 geçiş → 1**

İSRAFLI

```
def buyukten_kucuge(puanlar):
    return list(reversed(sorted(puanlar)))
```

VERİMLİ

```
def buyukten_kucuge(puanlar):
    return sorted(puanlar, reverse=True)
```

ÖLÇÜM süre 2,9 ms → 2,8 ms · bellek 312,6 KB → 234,4 KB · aynı sonuç ✓

Fark ölçülemeyecek kadar küçük: reversed() kopyası sıralamanın yanında önemsizdir. reverse=True ayrıca eşit elemanların sırasını korur.

Kaynak: Ruff C413 unnecessary-call-around-sorted · docs.astral.sh/ruff/rules/unnecessary-call-around-sorted/

YK064 bilgi Motor düzeltir — 2,42× hızlı

Kopyalamaktan başka iş yapmayan kurgu

[x for x in y] yalnızca kopyalar. list(y), set(y) ya da dict(y) aynı işi C hızında yapar.

NEDEN ENERJİ HARCAR?
Kurgu her eleman için Python döngüsü çalıştırır; list(y) kopyayı C ile yapar.

NE ZAMAN SORUN DEĞİL?
Eleman dönüştürülüyorsa kurgu doğrudur.

YAPILAN İŞ **Python döngüsü → C kopyası**

İSRAFLI

```
def kopya(liste):
    return [x for x in liste]
```

VERİMLİ

```
def kopya(liste):
    return list(liste)
```

ÖLÇÜM süre 411 µs → 170 µs · bellek 434,0 KB → 390,7 KB · aynı sonuç ✓

Motorun düzeltmesi de aynı sonucu verdi (2,39×).

Kaynak: Ruff C416 unnecessary-comprehension · docs.astral.sh/ruff/rules/unnecessary-comprehension/

YK065 bilgi Motor düzeltir **1.089× az bellek**

sum(), min() ya da max() içinde liste

Köşeli parantezli liste önce bütün elemanları bellekte tutar. Üreteç kullanın: sum(x for x in y).

NEDEN ENERJİ HARCAR?
Liste önce bütün elemanları bellekte tutar; üreteç elemanları tek tek verir ve bellek sabit kalır.

NE ZAMAN SORUN DEĞİL?
Çok küçük girdilerde liste biraz daha hızlı olabilir; fark bellekte büyüktür.

YAPILAN İŞ `O(n) bellek → O(1)`

İSRAFLI

```
def toplam_tutar(siparisler):
    return sum([s["tutar"] for s in siparisler])
```

VERİMLİ

```
def toplam_tutar(siparisler):
    return sum(s["tutar"] for s in siparisler)
```

ÖLÇÜM süre 1,0 ms → 1,6 ms · bellek 434,0 KB → 408 B · aynı sonuç ✓

Üreteç bellekte yer tutmaz ama CPython'da liste kurgusundan biraz yavaş çalışabilir; kazanç bellektedir.

Motorun düzeltmesi de aynı sonucu verdi (0,66×).

Kaynak: Pylint R1728 consider-using-generator · `pylint.readthedocs.io/en/stable/user_guide/messages/refactor/consider-using-generator.html`

YK066 bilgi Motor düzeltir **1,80× hızlı**

Değişmez değer (literal) etrafında gereksiz çağrı

list([...]), set([...]), tuple([...]) önce bir liste kurup sonra kopyalar. Doğrudan [...], {...} ya da (...) yazın.

NEDEN ENERJİ HARCAR?
Önce geçici bir liste oluşturulup sonra kopyalanır; literal sonucu doğrudan oluşturur.

NE ZAMAN SORUN DEĞİL?
Fark küçüktür ama kodu da sadeleştirir.

YAPILAN İŞ `2 nesne → 1`

İSRAFLI

```
def izinli_roller():
    return set(["yonetici", "editor", "okur"])
```

VERİMLİ

```
def izinli_roller():
    return {"yonetici", "editor", "okur"}
```

ÖLÇÜM süre 137 µs → 76 µs · bellek 219,7 KB → 219,6 KB · aynı sonuç ✓

Motorun düzeltmesi de aynı sonucu verdi (1,84×).

Kaynak: Ruff C405 / C406 / C409 / C410 / C411 · `docs.astral.sh/ruff/rules/unnecessary-literal-within-list-call/`

YK067 bilgi Öneri verir **216× az bellek**

Saymak için len(list(...))

len(list(x)) yalnızca saymak için bütün elemanları kopyalar. Dizi zaten varsa len(x), üreteçse sum(1 for _ in x) kullanın.

NEDEN ENERJİ HARCAR?
Saymak için bütün elemanların kopyası bellekte oluşturulur.

NE ZAMAN SORUN DEĞİL?
Liste sonra başka işler için de kullanılıyorsa fark yoktur.

YAPILAN İŞ `O(n) bellek → O(1)`

İSRAFLI

```
def satir_sayisi(dosya):
    return len(list(dosya))
```

VERİMLİ

```
def satir_sayisi(dosya):
    return sum(1 for _ in dosya)
```

ÖLÇÜM süre 7,1 ms → 7,0 ms · bellek 8,6 MB → 40,7 KB · aynı sonuç ✓

Süre benzer; asıl kazanç bellektedir: satırlar sayılırken listede tutulmaz.

Kaynak: Python belgeleri: len() · `docs.python.org/3/library/functions.html#len`

YK106 bilgi Öneri verir 1,21× hızlı

Kontrol edip kümeden silme

if x in s: s.remove(x) iki kez arama yapar. s.discard(x) tek aramada siler.

NEDEN ENERJİ HARCAR?
in ve remove ayrı ayrı karma hesaplar; discard tek seferde siler.

NE ZAMAN SORUN DEĞİL?
Eleman yoksa hata almak isteniyorsa remove kullanılır.

YAPILAN İŞ `2 arama → 1`

İSRAFLI

```
def cikis_yap(oturumlar, kullanici):
    if kullanici in oturumlar:
        oturumlar.remove(kullanici)
```

VERİMLİ

```
def cikis_yap(oturumlar, kullanici):
    oturumlar.discard(kullanici)
```

ÖLÇÜM süre 2,8 ms → 2,3 ms · bellek 169,0 KB → 169,0 KB · aynı sonuç ✓

Kaynak: Ruff FURB132 check-and-remove-from-set · `docs.astral.sh/ruff/rules/check-and-remove-from-set/`

YK107 bilgi Öneri verir 11,0× hızlı

Ters çevirmek için kopya

L = L[::-1] yeni bir liste oluşturur. Listeyi yerinde çevirmek için L.reverse() kullanın.

NEDEN ENERJİ HARCAR?
[::-1] yeni bir liste ayırır; reverse() yerinde çevirir.

NE ZAMAN SORUN DEĞİL?
Özgün listenin değişmemesi gerekiyorsa kopya gereklidir.

YAPILAN İŞ `kopya → yerinde`

İSRAFLI

```
def ters_cevir(adimlar):
    adimlar = adimlar[::-1]
    return adimlar
```

VERİMLİ

```
def ters_cevir(adimlar):
    adimlar.reverse()
    return adimlar
```

ÖLÇÜM süre 910 µs → 83 µs · bellek 781,2 KB → 0 B · aynı sonuç ✓

İyi sürüm listeyi yerinde çevirir; çağıran taraftaki liste de değişir. Özgün sıra başka yerde gerekiyorsa kopya şarttır.

Kaynak: Ruff FURB187 list-reverse-copy · `docs.astral.sh/ruff/rules/list-reverse-copy/`

YK108 bilgi Öneri verir 1,20× hızlı

Tek elemanlı koleksiyonda arama

x in [y] yalnızca karşılaştırma için bir liste oluşturur. x == y yazın.

NEDEN ENERJİ HARCAR?
Karşılaştırma için geçici bir liste oluşturulur.

NE ZAMAN SORUN DEĞİL?
NaN gibi kendine eşit olmayan değerlerde 'in' kimlik kontrolü de yapar; == farklı sonuç verebilir.

YAPILAN İŞ `liste + arama → karşılaştırma`

İSRAFLI

```
def yonetici_mi(rol):
    return rol in ["yonetici"]
```

VERİMLİ

```
def yonetici_mi(rol):
    return rol == "yonetici"
```

ÖLÇÜM süre 2,1 ms → 1,8 ms · bellek 434,0 KB → 434,0 KB · aynı sonuç ✓

Kaynak: Ruff FURB171 single-item-membership-test · `docs.astral.sh/ruff/rules/single-item-membership-test/`

YK109 bilgi Öneri verir 7,41× hızlı

Bit saymak için metne çevirme

bin(x).count('1') sayıyı önce metne çevirir. Python 3.10+ ile x.bit_count() doğrudan sayar.

NEDEN ENERJİ HARCAR?
bin() sayıyı metne çevirir, count metni tarar; bit_count tek işlemci komutuyla (popcount) sayar.

NE ZAMAN SORUN DEĞİL?
Python 3.10 öncesinde bit_count yoktur.

YAPILAN İŞ `metin → tek komut`

İSRAFLI

```
def acik_bit_sayisi(izinler):
    return bin(izinler).count("1")
```

VERİMLİ

```
def acik_bit_sayisi(izinler):
    return izinler.bit_count()
```

ÖLÇÜM süre 15,3 ms → 2,1 ms · bellek 434,0 KB → 434,0 KB · aynı sonuç ✓

Kaynak: Ruff FURB161 bit-count · `docs.astral.sh/ruff/rules/bit-count/`

YK110 bilgi Öneri verir **1,41× hızlı**

math.log(x, 2) ya da math.log(x, 10)

Taban verilen log iki logaritma ve bir bölme yapar. math.log2(x) ve math.log10(x) tek adımda ve daha kesin hesaplar.

NEDEN ENERJİ HARCAR?
log(x, 2) iki logaritma ve bir bölme yapar; log2 tek fonksiyondur ve daha kesindir.

NE ZAMAN SORUN DEĞİL?
Taban değişkense log(x, taban) gerekir.

YAPILAN İŞ **2 log + bölme → 1**

İSRAFLI

```
import math


def bit_uzunlugu(n):
    return math.log(n, 2)
```

VERİMLİ

```
import math


def bit_uzunlugu(n):
    return math.log2(n)
```

ÖLÇÜM süre 3,8 ms → 2,7 ms · bellek 1,6 MB → 1,6 MB · aynı sonuç ✓

İyi sürüm daha kesindir: math.log(1000, 10) gibi değerlerde son basamakta fark olabilir; karşılaştırma bu küçük farka izin verir.

Kaynak: Ruff FURB163 redundant-log-base · docs.astral.sh/ruff/rules/redundant-log-base/

YK112 bilgi Öneri verir **1,26× hızlı**

Art arda append()

Aynı listeye art arda append() çağırmak yerine tek bir extend([...]) kullanın.

NEDEN ENERJİ HARCAR?
Her append ayrı bir metot çağrısıdır; extend elemanları tek çağrıda ekler.

NE ZAMAN SORUN DEĞİL?
Eklemeler arasında başka iş yapılıyorsa ayrı append doğaldır.

YAPILAN İŞ **n çağrı → 1**

İSRAFLI

```
def basliklar():
    satirlar = []
    satirlar.append("Ad")
    satirlar.append("Soyad")
    satirlar.append("Numara")
    return satirlar
```

VERİMLİ

```
def basliklar():
    return ["Ad", "Soyad", "Numara"]
```

ÖLÇÜM süre 85 µs → 68 µs · bellek 90,3 KB → 90,3 KB · aynı sonuç ✓

Kaynak: Ruff FURB113 repeated-append · docs.astral.sh/ruff/rules/repeated-append/

YK113 bilgi Motor düzeltir **1,37× hızlı**

Üreteci list(), set() ya da dict() ile sarma

list(x for x in y) önce bir üreteç kurar, sonra onu tüketir. Doğrudan liste, küme ya da sözlük kurgusu yazın.

NEDEN ENERJİ HARCAR?
Üreteç nesnesi kurulur ve her eleman için çerçeve geçişi yapılır; kurgu sonucu doğrudan oluşturur.

NE ZAMAN SORUN DEĞİL?
Fark küçüktür ama kodu da sadeleştirir.

YAPILAN İŞ **üreteç + kopya → kurgu**

İSRAFLI

```
def buyuk_harfli(adlar):
    return list(a.upper() for a in adlar)
```

VERİMLİ

```
def buyuk_harfli(adlar):
    return [a.upper() for a in adlar]
```

ÖLÇÜM süre 1,2 ms → 864 µs · bellek 1,1 MB → 1,1 MB · aynı sonuç ✓

Motorun düzeltmesi de aynı sonucu verdi (1,55×).

Kaynak: Ruff C400 / C401 / C402 · docs.astral.sh/ruff/rules/unnecessary-generator-list/

---

04 Metin ve düzenli ifadeler

7 kural

YK010 orta Öneri verir

1,20× hızlı

Döngüde metin birleştirme

Metne döngüde += ile eklemek her adımda yeni bir metin oluşturabilir; uzun metinlerde iş karesel büyür. Parçaları listede toplayıp sonunda ''.join() kullanın.

NEDEN ENERJİ HARCAR?

Python metinleri değiştirilemez; += her seferinde yeni bir metin ayırıp eskisini kopyalayabilir. CPython bazen yerinde büyütür ama bu garanti değildir ve metne başka bir referans varsa çalışmaz.

NE ZAMAN SORUN DEĞİL?

Birkaç parça birleştiriliyorsa ya da döngü kısaysa fark önemsizdir.

YAPILAN İŞ `O(n²) → O(n)`

İSRAFLI

```
def rapor_metni(satirlar):
    metin = ""
    for s in satirlar:
        metin += f"{s['ad']}: {s['tutar']} TL\n"
    return metin
```

VERİMLİ

```
def rapor_metni(satirlar):
    return "".join(f"{s['ad']}: {s['tutar']} TL\n" for s in
  satirlar)
```

ÖLÇÜM süre 783 µs → 651 µs · bellek 93,2 KB → 505,3 KB · aynı sonuç ✓

CPython, başka referansı olmayan metne += eklerken çoğu zaman yerinde büyütür; bu yüzden fark burada küçük çıkabilir. Bu bir garanti değildir; PyPy ve başka uygulamalarda yoktur.

Kaynak: Pylint R1713 consider-using-join · `pylint.readthedocs.io/en/stable/user_guide/messages/refactor/consider-using-join.html`

YK069 orta Öneri verir

fark yok

Düz metin için düzenli ifade

Kalıpta özel karakter yok; düzenli ifade motoru gereksiz. s.startswith(), in, str.replace() ya da str.split() aynı işi daha az emekle yapar.

NEDEN ENERJİ HARCAR?

Düzenli ifade motoru deseni derler ve bir durum makinesiyle eşleştirir; düz metin için str yöntemleri doğrudan C döngüsüyle çalışır.

NE ZAMAN SORUN DEĞİL?

Desen ileride gerçek bir düzenli ifadeye dönüşecekse regex tutarlılık için kalabilir.

YAPILAN İŞ `regex motoru → doğrudan karşılaştırma`

İSRAFLI

```
import re


def virgulle_bol(metin):
    return re.split(",", metin)
```

VERİMLİ

```
def virgulle_bol(metin):
    return metin.split(",")
```

ÖLÇÜM süre 697 µs → 656 µs · bellek 1,1 MB → 1,1 MB · aynı sonuç ✓

Kaynak: Python belgeleri: str yöntemleri · `docs.python.org/3/library/stdtypes.html#string-methods`

YK070 bilgi Öneri verir

fark yok

Döngüde derlenmemiş düzenli ifade

re.match/re.search her çağrıda kalıbı önbellekte arar. Döngüden önce re.compile ile derleyip derlenmiş nesneyi kullanın.

NEDEN ENERJİ HARCAR?

re modülü derlenmiş desenleri önbellekte tutar ama her çağrıda önbellek araması ve tür kontrolü yapar; derlenmiş nesne bu adımı atlar.

NE ZAMAN SORUN DEĞİL?

Önbellek olduğu için fark küçüktür; sıcak döngülerde anlamlıdır.

YAPILAN İŞ `n önbellek araması → 0`

İSRAFLI

```
import re


def posta_kodlari(satirlar):
    return [re.search(r"\d{5}", s) for s in satirlar]
```

VERİMLİ

```
import re

POSTA_KODU = re.compile(r"\d{5}")


def posta_kodlari(satirlar):
    return [POSTA_KODU.search(s) for s in satirlar]
```

ÖLÇÜM süre 14,2 ms → 13,2 ms · bellek 2,5 MB → 2,5 MB · aynı sonuç ✓

Kaynak: Python belgeleri: re.compile · `docs.python.org/3/library/re.html#re.compile`

YK071 bilgi Öneri verir **1,60× hızlı**

Art arda startswith() ya da endswith()

s.startswith('a') or s.startswith('b') her önek için ayrı bir metot çağrısı yapar. Önekleri bir demette verin: s.startswith(('a', 'b')).

NEDEN ENERJİ HARCAR?
Her startswith çağrısı ayrı bir metot araması ve çağrısıdır. Demetle verilen önekler tek çağrıda, C içinde sırayla denenir.

NE ZAMAN SORUN DEĞİL?
Koşullar farklı metinleri kontrol ediyorsa ya da arada başka bir iş yapılıyorsa birleştirilemez.

YAPILAN İŞ **k çağrı → 1**

İSRAFLI

```
def resim_mi(dosya_adi):
    return dosya_adi.endswith(".png") or
  dosya_adi.endswith(".jpg") or dosya_adi.endswith(".webp")
```

VERİMLİ

```
def resim_mi(dosya_adi):
    return dosya_adi.endswith((".png", ".jpg", ".webp"))
```

ÖLÇÜM süre 3,5 ms → 2,2 ms · bellek 434,0 KB → 434,0 KB · aynı sonuç ✓

Kaynak: Ruff PIE810 multiple-starts-ends-with · docs.astral.sh/ruff/rules/multiple-starts-ends-with/

YK072 bilgi Öneri verir **fark yok**

İçerme kontrolü için find()

find() konumu hesaplar; yalnızca içerip içermediği soruluyorsa 'alt in metin' daha açık ve hızlıdır.

NEDEN ENERJİ HARCAR?
find konumu hesaplar ve sonuç ayrıca karşılaştırılır; 'in' doğrudan içerme sorusunu yanıtlar ve daha okunaklıdır.

NE ZAMAN SORUN DEĞİL?
Konum gerçekten gerekiyorsa find doğru araçtır.

YAPILAN İŞ **konum + karşılaştırma → içerme**

İSRAFLI

```
def hata_var_mi(log):
    return log.find("ERROR") != -1
```

VERİMLİ

```
def hata_var_mi(log):
    return "ERROR" in log
```

ÖLÇÜM süre 738 µs → 727 µs · bellek 28 B → 10 B · aynı sonuç ✓

Hız aynı: ikisi de aynı C aramasını yapar. Kazanç okunabilirliktedir; Python belgeleri de içerme için 'in' önerir.

Kaynak: Python belgeleri: str.find · docs.python.org/3/library/stdtypes.html#str.find

---

YK073 bilgi Öneri verir **215× az bellek**

Bütün dosyayı okuyup satırlara bölme

read().split('\n') ya da read().splitlines() dosyanın tamamını ve satır listesini birlikte bellekte tutar. Dosyayı satır satır gezin.

NEDEN ENERJİ HARCAR?
read() dosyanın tamamını tek bir metin olarak belleğe alır, split ise ikinci bir kopya olarak satır listesini kurar. Dosya nesnesini gezmek tamponla satır satır okur; bellek dosya boyutundan bağımsız kalır. Python öğreticisi bunu 'bellek dostu, hızlı ve basit' diye önerir.

NE ZAMAN SORUN DEĞİL?
Küçük yapılandırma dosyalarında ya da satırlara rastgele erişim gerektiğinde bütün dosyayı okumak sorun değildir.

YAPILAN İŞ **O(dosya) bellek → O(satır)**

İSRAFLI

```
def dolu_satir_sayisi(yol):
    with open(yol) as f:
        satirlar = f.read().split("\n")
    return sum(1 for s in satirlar if s.strip())
```

VERİMLİ

```
def dolu_satir_sayisi(yol):
    with open(yol) as f:
        return sum(1 for s in f if s.strip())
```

ÖLÇÜM süre 9,9 ms → 12,1 ms · bellek 9,6 MB → 45,5 KB · aynı sonuç ✓

Süre benzer; asıl kazanç bellektedir: iyi sürüm dosyayı satır satır okur.

Kaynak: Python belgeleri: Dosya nesnelerinin yöntemleri (öğretici 7.2.1) · docs.python.org/3/tutorial/inputoutput.html#methods-of-file-objects

---

YK074 `bilgi` `Öneri verir` **1,69× hızlı**

Önek kontrolü için dilimleme

s[:3] == 'abc' önce yeni bir metin keser. s.startswith('abc') ya da s.endswith('abc') kopya oluşturmaz.

NEDEN ENERJİ HARCAR?

Dilimleme yeni bir metin nesnesi oluşturur; startswith karşılaştırmayı kopyasız yapar.

NE ZAMAN SORUN DEĞİL?

Kısa önekler için fark küçüktür; okunabilirlik kazancı vardır.

YAPILAN İŞ `kopya + karşılaştırma → karşılaştırma`

İSRAFLI

```
def https_mi(adres):
    return adres[:8] == "https://"
```

VERİMLİ

```
def https_mi(adres):
    return adres.startswith("https://")
```

ÖLÇÜM süre 3,7 ms → 2,2 ms · bellek 434,0 KB → 434,0 KB · aynı sonuç ✓

Kaynak: Python belgeleri: str.startswith · `docs.python.org/3/library/stdtypes.html#str.startswith`

---

05 Veri bilimi (pandas, NumPy)

17 kural

YK007 yüksek Öneri verir

914× hızlı

Pandas tablosunu satır satır gezme

iterrows, itertuples ve axis=1 ile apply her satırı yavaş bir Python döngüsünde işler. Sütun işlemleri, merge ve groupby gibi vektörel pandas/numpy işlemlerini kullanın; yapılan iş genellikle onlarca kat azalır.

NEDEN ENERJİ HARCAR?

iterrows her satır için ayrı bir Series nesnesi kurar ve Python yorumlayıcısında döner. Vektörel işlemler aynı hesabı derlenmiş kodla, veri üzerinde tek geçişte yapar.

NE ZAMAN SORUN DEĞİL?

Satır başına dış bir servis çağrılıyorsa (API) döngünün maliyeti zaten o çağrıdadır.

YAPILAN İŞ **Python'da n adım → C'de vektörel**

İSRAFLI

```
def toplam_ciro(satislar):
    toplam = 0
    for _, satir in satislar.iterrows():
        toplam += satir["adet"] * satir["fiyat"]
    return toplam
```

VERİMLİ

```
def toplam_ciro(satislar):
    return (satislar["adet"] * satislar["fiyat"]).sum()
```

ÖLÇÜM süre 34,4 ms → 38 µs · bellek 50,9 KB → 50,1 KB · aynı sonuç ✓

Kaynak: pandas belgeleri: Enhancing performance · pandas.pydata.org/docs/user_guide/enhancingperf.html

---

YK018 yüksek Öneri verir

1,26× hızlı

Döngüde dizi ya da tablo büyütme

pd.concat ve np.append/concatenate/vstack her çağrıda bütün veriyi yeniden kopyalar. Parçaları listede toplayıp döngüden sonra tek seferde birleştirin.

NEDEN ENERJİ HARCAR?

NumPy dizileri ve DataFrame'ler bitişik bellek bloklarıdır; her concat/append yeni blok ayırıp bütün veriyi kopyalar. Döngüde bu karesel kopyalamaya döner.

NE ZAMAN SORUN DEĞİL?

Yalnızca birkaç parça birleştiriliyorsa fark küçüktür.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
import pandas as pd


def dosyalari_birlestir(dosya_yollari):
    tablo = pd.DataFrame()
    for yol in dosya_yollari:
        parca = pd.read_csv(yol)
        tablo = pd.concat([tablo, parca])
    return tablo
```

VERİMLİ

```
import pandas as pd


def dosyalari_birlestir(dosya_yollari):
    return pd.concat([pd.read_csv(yol, usecols=["ad", "puan"])
  for yol in dosya_yollari])
```

ÖLÇÜM süre 27,7 ms → 22,0 ms · bellek 936,7 KB → 1,1 MB · aynı sonuç ✓

Kaynak: pandas belgeleri: concat · NumPy belgeleri: append · pandas.pydata.org/docs/user_guide/merging.html

---

YK027 yüksek Öneri verir **2.450× hızlı**

NumPy dizisinde eleman eleman Python döngüsü

NumPy dizisini Python döngüsüyle gezmek vektörel işlemlerden onlarca kat yavaştır. Diziyle doğrudan işlem yapın: a * 2, np.where, np.sum.

NEDEN ENERJİ HARCAR?
Her eleman erişimi bir Python nesnesi oluşturur ve yorumlayıcıda döner; vektörel işlem aynı hesabı derlenmiş kodla ve SIMD komutlarıyla yapar.

NE ZAMAN SORUN DEĞİL?
Her adım bir öncekine bağlıysa (özyinelemeli filtre) vektörleştirmek zor olabilir; Numba gibi araçlar düşünülebilir.

YAPILAN İŞ **Python'da n adım → C'de vektörel**

İSRAFLI

```
import numpy as np


def normalize(dizi):
    sonuc = np.zeros(len(dizi))
    for i in range(len(dizi)):
        sonuc[i] = (dizi[i] - dizi.mean()) / dizi.std()
    return sonuc
```

VERİMLİ

```
def normalize(dizi):
    return (dizi - dizi.mean()) / dizi.std()
```

ÖLÇÜM süre 39,9 ms → 16 µs · bellek 48,2 KB → 48,1 KB · aynı sonuç ✓

Kaynak: NumPy belgeleri: Broadcasting ve vektörleştirme · `numpy.org/doc/stable/user/basics.broadcasting.html`

YK075 yüksek Öneri verir **ölçülmedi**

Döngüde DataFrame.append

df = df.append(...) her adımda bütün tabloyu kopyalar; pandas 2.0'da kaldırıldı. Satırları listede toplayıp döngüden sonra tek bir DataFrame oluşturun.

NEDEN ENERJİ HARCAR?
Her append bütün tabloyu yeni bir DataFrame'e kopyalar; n satır için toplam kopya karesel büyür. pandas 2.0'da kaldırıldı.

NE ZAMAN SORUN DEĞİL?
Yok; pandas 2.0 ve sonrasında çalışmaz.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
import pandas as pd


def tabloya_ekle(satirlar):
    df = pd.DataFrame(columns=["ad", "puan"])
    for s in satirlar:
        df = df.append(s, ignore_index=True)
    return df
```

VERİMLİ

```
import pandas as pd


def tabloya_ekle(satirlar):
    return pd.DataFrame(satirlar, columns=["ad", "puan"])
```

ÖLÇÜLMEDİ DataFrame.append pandas 2.0'da kaldırıldı; kötü sürüm güncel pandas'ta çalışmaz. Aynı kalıbın çalışan hâli YK076'da ölçüldü.

Kaynak: pandas 2.0 sürüm notları: DataFrame.append kaldırıldı · `pandas.pydata.org/docs/whatsnew/v2.0.0.html`

YK076 yüksek Öneri verir **744× hızlı**

Döngüde df.loc ile satır ekleme

df.loc[len(df)] = ... her eklemede tabloyu yeniden ayırır ve kopyalar. Satırları listede toplayıp sonunda DataFrame'e çevirin.

NEDEN ENERJİ HARCAR?
Her satır ekleme, tabloyu büyütmek için yeniden bellek ayırma ve kopyalama yapar.

NE ZAMAN SORUN DEĞİL?
Birkaç satır ekleniyorsa fark küçüktür.

YAPILAN İŞ **O(n²) → O(n)**

İSRAFLI

```
import pandas as pd


def tablo_olustur(satirlar):
    df = pd.DataFrame(columns=["ad", "puan"])
    for s in satirlar:
        df.loc[len(df)] = [s["ad"], s["puan"]]
    return df
```

VERİMLİ

```
import pandas as pd


def tablo_olustur(satirlar):
    return pd.DataFrame(satirlar, columns=["ad", "puan"])
```

ÖLÇÜM süre 91,9 ms → 124 µs · bellek 71,9 KB → 22,3 KB · aynı sonuç ✓

Kaynak: pandas belgeleri: Merge, join, concatenate · `pandas.pydata.org/docs/user_guide/merging.html`

YK077 orta Öneri verir 90,4× hızlı

apply() ile vektörleştirilebilir işlem

apply(lambda x: x * 2) her eleman için Python fonksiyonu çağırır. Sütunla doğrudan işlem yapın: df['a'] * 2.

NEDEN ENERJİ HARCAR?

apply her eleman için Python lambda'sını çağırır; sütun işlemi aynı hesabı C/NumPy ile tek geçişte yapar.

NE ZAMAN SORUN DEĞİL?

Hesap gerçekten satır satır Python mantığı istiyorsa apply gerekebilir.

YAPILAN İŞ `n Python çağrısı → vektörel`

İSRAFLI

```
def kdv_ekle(df):
    df["kdvli"] = df["fiyat"].apply(lambda x: x * 1.2)
    return df
```

VERİMLİ

```
def kdv_ekle(df):
    df["kdvli"] = df["fiyat"] * 1.2
    return df
```

ÖLÇÜM süre 6,3 ms → 70 µs · bellek 5,1 MB → 788,2 KB · aynı sonuç ✓

Kaynak: pandas belgeleri: Enhancing performance · `pandas.pydata.org/docs/user_guide/enhancingperf.html`

YK078 orta Öneri verir 1,43× hızlı

apply() ile metin işlemi

apply(lambda x: x.lower()) her hücre için Python çağrısı yapar. .str.lower() gibi .str yöntemlerini kullanın.

NEDEN ENERJİ HARCAR?

.str yöntemleri toplu çalışır ve eksik değerleri (NaN) doğru işler; apply her hücre için Python çağrısı yapar.

NE ZAMAN SORUN DEĞİL?

İşlem .str ile ifade edilemiyorsa apply gerekebilir.

YAPILAN İŞ `n Python çağrısı → toplu`

İSRAFLI

```
def kucuk_harf(df):
    df["eposta"] = df["eposta"].apply(lambda x: x.lower())
    return df
```

VERİMLİ

```
def kucuk_harf(df):
    df["eposta"] = df["eposta"].str.lower()
    return df
```

ÖLÇÜM süre 5,4 ms → 3,8 ms · bellek 5,4 MB → 5,4 MB · aynı sonuç ✓

Kaynak: pandas belgeleri: Working with text data · `pandas.pydata.org/docs/user_guide/text.html`

YK079 bilgi Öneri verir 205× hızlı

np.vectorize ile hız beklentisi

np.vectorize kolaylık içindir; NumPy belgelerine göre esasen bir for döngüsüdür. Gerçek vektörel işlemler ya da np.where kullanın.

NEDEN ENERJİ HARCAR?

np.vectorize Python fonksiyonunu her eleman için çağırır; NumPy belgelerine göre esasen bir for döngüsüdür.

NE ZAMAN SORUN DEĞİL?

Okunabilirlik için kabul edilebilir ama hız beklenmemelidir.

YAPILAN İŞ `n Python çağrısı → vektörel`

İSRAFLI

```
import numpy as np


def notlari_harflendir(notlar):
    harf = np.vectorize(lambda n: "AA" if n >= 90 else "BB")
    return harf(notlar)
```

VERİMLİ

```
import numpy as np


def notlari_harflendir(notlar):
    return np.where(notlar >= 90, "AA", "BB")
```

ÖLÇÜM süre 7,8 ms → 38 µs · bellek 1,1 MB → 441,2 KB · aynı sonuç ✓

Kaynak: NumPy belgeleri: numpy.vectorize · `numpy.org/doc/stable/reference/generated/numpy.vectorize.html`

YK080 orta Öneri verir **13,8× hızlı**

NumPy dizisinde yerleşik sum(), min() ya da max()

Python'un sum/min/max fonksiyonları diziyi eleman eleman Python nesnesine çevirerek gezer. arr.sum(), arr.min(), arr.max() C hızında çalışır.

NEDEN ENERJİ HARCAR?

Python'un sum'ı diziyi eleman eleman NumPy skalerine çevirerek yorumlayıcıda toplar; arr.sum() C döngüsüyle ve SIMD ile toplar.

NE ZAMAN SORUN DEĞİL?

Dizi çok küçükse (birkaç eleman) fark yoktur.

YAPILAN İŞ `Python döngüsü → C döngüsü`

İSRAFLI

```
import numpy as np


def ortalama(n):
    olcumler = np.random.rand(n)
    return sum(olcumler) / n
```

VERİMLİ

```
import numpy as np


def ortalama(n):
    olcumler = np.random.rand(n)
    return olcumler.sum() / n
```

ÖLÇÜM süre 11,6 ms → 847 µs · bellek 1,5 MB → 1,5 MB · aynı sonuç ✓

Kaynak: NumPy belgeleri: ndarray.sum · `numpy.org/doc/stable/reference/generated/numpy.ndarray.sum.html`

YK081 orta Öneri verir **2,00× hızlı**

Döngüde sabit dizi ya da tensör oluşturma

Döngünün her adımında aynı içerikle np.array/torch.tensor oluşturuluyor. Diziyi döngüden önce bir kez oluşturun.

NEDEN ENERJİ HARCAR?

Her adımda bellek ayırma ve veri kopyalama yapılır; sabit dizi bir kez oluşturulabilir.

NE ZAMAN SORUN DEĞİL?

Dizi döngüde değiştiriliyorsa her adımda yeni dizi gerekebilir.

YAPILAN İŞ `n ayırma → 1`

İSRAFLI

```
import numpy as np


def donustur(noktalar):
    sonuc = []
    for p in noktalar:
        matris = np.array([[0, -1], [1, 0]])
        sonuc.append(matris @ p)
    return sonuc
```

VERİMLİ

```
import numpy as np

DONDUR = np.array([[0, -1], [1, 0]])


def donustur(noktalar):
    return [DONDUR @ p for p in noktalar]
```

ÖLÇÜM süre 7,6 ms → 3,8 ms · bellek 666,4 KB → 666,3 KB · aynı sonuç ✓

Kaynak: NumPy belgeleri: Array creation · `numpy.org/doc/stable/user/basics.creation.html`

YK082 bilgi Öneri verir **1,20× yavaş**

Zincirleme indeksleme ile atama

df['sütun'][i] = x önce bir ara kopya oluşturabilir ve değişikliği kaybedebilir. df.loc[i, 'sütun'] = x kullanın.

NEDEN ENERJİ HARCAR?

İlk indeksleme bir kopya döndürebilir; atama kopyaya yapılır, asıl tablo değişmez (SettingWithCopyWarning) ve ek kopya bellek harcar.

NE ZAMAN SORUN DEĞİL?

pandas 3.0 copy-on-write ile davranış değişir; .loc her sürümde doğrudur.

YAPILAN İŞ `ara kopya → yerinde atama`

İSRAFLI

```
def indirim_uygula(df, satir):
    df["fiyat"][satir] = df["fiyat"][satir] * 0.9
    return df
```

VERİMLİ

```
def indirim_uygula(df, satir):
    df.loc[satir, "fiyat"] = df.loc[satir, "fiyat"] * 0.9
    return df
```

ÖLÇÜM süre 58 µs → 70 µs · bellek 3,4 KB → 10,7 KB · aynı sonuç ✓

Bu kuralın asıl amacı doğruluktur: zincirleme atama kopya üzerinde çalışırsa değişiklik kaybolur. pandas 3.0'da (copy-on-write) kötü sürüm tabloyu hiç değiştirmez.

Kaynak: pandas belgeleri: Returning a view versus a copy · `pandas.pydata.org/docs/user_guide/indexing.html#returning-a-view-versus-a-copy`

YK083 bilgi Öneri verir **1,96× hızlı**

groupby().apply() ile toplama

groupby().apply() her grup için Python fonksiyonu çağırır. Toplama için agg(), satır başına sonuç için transform() kullanın.

NEDEN ENERJİ HARCAR?

apply her grup için Python fonksiyonu çağırıp sonuçları birleştirir; agg ve transform optimize edilmiş yolları kullanır.

NE ZAMAN SORUN DEĞİL?

Grup başına gerçekten karmaşık bir hesap gerekiyorsa apply gerekebilir.

YAPILAN İŞ **k Python çağrısı → optimize toplama**

İSRAFLI

```
def sehir_ortalamasi(df):
    return df.groupby("sehir").apply(lambda g:
  g["tutar"].mean())
```

VERİMLİ

```
def sehir_ortalamasi(df):
    return df.groupby("sehir")["tutar"].mean()
```

ÖLÇÜM süre 3,5 ms → 1,8 ms · bellek 2,3 MB → 1,8 MB · aynı sonuç ✓

Kaynak: pandas belgeleri: Group by (agg, transform) · pandas.pydata.org/docs/user_guide/groupby.html

---

YK084 bilgi Öneri verir **1,31× hızlı**

read_csv ile bütün sütunları okuma

usecols verilmezse bütün sütunlar okunur ve genellikle bellek dostu olmayan türlerle saklanır. usecols ve dtype belirtin.

NEDEN ENERJİ HARCAR?

Bütün sütunlar ayrıştırılır ve metin sütunları Python nesnesi (object) olarak saklanır; bellek gereğin katları olabilir.

NE ZAMAN SORUN DEĞİL?

Dosya küçükse ya da bütün sütunlar gerekiyorsa fark yoktur.

YAPILAN İŞ **bütün sütunlar → gerekenler**

İSRAFLI

```
import pandas as pd


def satislari_oku(yol):
    df = pd.read_csv(yol)
    return df["tutar"].sum()
```

VERİMLİ

```
import pandas as pd


def satislari_oku(yol):
    df = pd.read_csv(yol, usecols=["tutar"], dtype={"tutar":
  "int64"})
    return df["tutar"].sum()
```

ÖLÇÜM süre 24,3 ms → 18,6 ms · bellek 7,0 MB → 2,0 MB · aynı sonuç ✓

Kaynak: pandas belgeleri: Scaling to large datasets · pandas.pydata.org/docs/user_guide/scale.html

---

YK085 bilgi Öneri verir **5,03× hızlı**

Büyük veri için CSV

CSV sıkıştırılmamış metindir; okurken her değer yeniden ayrıştırılır. Büyük tablolar için Parquet gibi sütunlu ve sıkıştırılmış biçim kullanın.

NEDEN ENERJİ HARCAR?

CSV her okumada metin ayrıştırma ister, türleri kaybeder ve sıkıştırılmaz. Parquet sütunlu, türlü ve sıkıştırılmıştır; okuması çok daha az iş ister.

NE ZAMAN SORUN DEĞİL?

Başka sistemlerle paylaşım için CSV gerekebilir; küçük dosyalarda fark önemsizdir.

YAPILAN İŞ **metin ayrıştırma → sütunlu okuma**

İSRAFLI

```
def disari_aktar(df):
    df.to_csv("buyuk_tablo.csv", index=False)
```

VERİMLİ

```
def disari_aktar(df):
    df.to_parquet("buyuk_tablo.parquet", index=False)
```

ÖLÇÜM süre 52,5 ms → 10,4 ms · bellek 7,5 MB → 16,4 KB · dosya 1,0 MB → 605,6 KB · aynı sonuç ✓

Çıktılar dosyadan geri okunarak karşılaştırıldı. Parquet ayrıca diskte daha az yer tutar ve sonraki okumalar çok daha hızlıdır.

Kaynak: Green Software Foundation: Compress stored data · pandas: Scaling to large datasets · pandas.pydata.org/docs/user_guide/scale.html#use-efficient-datatypes

---

YK086 bilgi Öneri verir **1,69× hızlı**

NumPy dizisini list() ile listeye çevirme

list(arr) her elemanı ayrı bir NumPy skalerine çevirir. arr.tolist() daha hızlıdır; çoğu zaman diziyle çalışmak en iyisidir.

NEDEN ENERJİ HARCAR?
list() her elemanı ayrı bir NumPy skaler nesnesine sarar; tolist() doğrudan Python sayılarına çevirir.

NE ZAMAN SORUN DEĞİL?
Dizi küçükse fark önemsizdir.

YAPILAN İŞ **NumPy skalerleri → Python sayıları**

İSRAFLI

```
import numpy as np


def listeye(n):
    dizi = np.arange(n)
    return list(dizi)
```

VERİMLİ

```
import numpy as np


def listeye(n):
    return np.arange(n).tolist()
```

ÖLÇÜM süre 2,7 ms → 1,6 ms · bellek 3,8 MB → 4,6 MB · aynı sonuç ✓

Kaynak: NumPy belgeleri: ndarray.tolist · numpy.org/doc/stable/reference/generated/numpy.ndarray.tolist.html

---

YK087 orta Öneri verir **20,5× hızlı**

Döngüde DataFrame filtreleme

Döngüde df[df['k'] == değer] her adımda bütün tabloyu tarar. groupby('k') tek geçişte bütün grupları verir.

NEDEN ENERJİ HARCAR?
Her filtre bütün tabloyu tarar ve bir boolean maske oluşturur; k grup için k tam geçiş yapılır. groupby tek geçişte gruplar.

NE ZAMAN SORUN DEĞİL?
Yalnızca bir iki değer için filtre yapılıyorsa fark küçüktür.

YAPILAN İŞ **k geçiş → 1**

İSRAFLI

```
def sehir_toplamlari(df, sehirler):
    sonuc = {}
    for s in sehirler:
        sonuc[s] = df[df["sehir"] == s]["tutar"].sum()
    return sonuc
```

VERİMLİ

```
def sehir_toplamlari(df, sehirler):
    toplamlar = df.groupby("sehir")["tutar"].sum()
    return {s: toplamlar.get(s, 0) for s in sehirler}
```

ÖLÇÜM süre 34,5 ms → 1,7 ms · bellek 146,9 KB → 1,8 MB · aynı sonuç ✓

Kaynak: pandas belgeleri: Group by · pandas.pydata.org/docs/user_guide/groupby.html

---

YK111 bilgi Öneri verir **28,2× az bellek**

Sabitlik kontrolü için nunique()

nunique() bütün farklı değerleri sayar. Serinin tek değerden oluştuğunu anlamak için (s == s.iloc[0]).all() ilk farkta durabilir.

NEDEN ENERJİ HARCAR?
nunique bütün değerlerin karma tablosunu kurar; sabitlik kontrolü ilk farklı değerde durabilir.

NE ZAMAN SORUN DEĞİL?
Farklı değer sayısı başka bir yerde de gerekiyorsa nunique doğrudur.

YAPILAN İŞ **tam karma tablosu → ilk farka kadar**

İSRAFLI

```
def tek_sehir_mi(df):
    return df["sehir"].nunique() == 1
```

VERİMLİ

```
def tek_sehir_mi(df):
    sehir = df["sehir"]
    return bool((sehir == sehir.iloc[0]).all())
```

ÖLÇÜM süre 2,9 ms → 2,8 ms · bellek 2,8 MB → 101,0 KB · aynı sonuç ✓

Bütün değerler aynı olduğu için iki sürüm de seriyi sonuna kadar tarar ve süre aynıdır. Kazanç bellektedir: nunique bütün değerlerin karma tablosunu kurar.

Kaynak: Ruff PD101 pandas-nunique-constant-series-check · docs.astral.sh/ruff/rules/pandas-nunique-constant-series-check/

---

06 Makine öğrenmesi

4 kural

YK088 yüksek Öneri verir

77,2× hızlı

Döngüde tek tek tahmin

Modeli her örnek için ayrı çağırmak toplu işlemenin (batch) verimini kaybettirir. Örnekleri toplayıp predict() ile birlikte gönderin.

NEDEN ENERJİ HARCAR?
Her predict çağrısı giriş doğrulama, bellek ayırma ve (GPU varsa) aktarım maliyeti öder; toplu çağrı bu sabit maliyeti bir kez öder ve vektörel hesaplar.

NE ZAMAN SORUN DEĞİL?
Gerçek zamanlı tek istek (ör. web isteği başına bir tahmin) için toplamak mümkün olmayabilir.

YAPILAN İŞ **n sabit maliyet → 1**

İSRAFLI

```
def siniflandir(model, ornekler):
    return [model.predict([x])[0] for x in ornekler]
```

VERİMLİ

```
def siniflandir(model, ornekler):
    return list(model.predict(ornekler))
```

ÖLÇÜM süre 10,1 ms → 132 µs · bellek 11,4 KB → 17,5 KB · aynı sonuç ✓

Kaynak: scikit-learn ve Keras belgeleri: toplu tahmin · scikit-learn.org/stable/computing/computational_performance.html

---

YK089 bilgi Öneri verir

ölçülmedi

Döngüde GPU'dan değer çekme

.item(), .cpu() ve .numpy() her çağrıda işlemciyi GPU'yu beklemeye zorlar (eşitleme). Değerleri birkaç adımda bir okuyun.

NEDEN ENERJİ HARCAR?
GPU işleri sıraya koyarak eşzamansız çalışır; .item() sonucun hazır olmasını beklemeye zorlar ve kuyruğu boşaltır.

NE ZAMAN SORUN DEĞİL?
Değeri her adımda gerçekten kullanmak gerekiyorsa (erken durdurma) kaçınılmazdır; seyrek okuyun.

YAPILAN İŞ **her adımda eşitleme → seyrek**

İSRAFLI

```
def egit(model, veri, kayip_fn, optimizer):
    kayiplar = []
    for x, y in veri:
        kayip = kayip_fn(model(x), y)
        kayip.backward()
        optimizer.step()
        kayiplar.append(kayip.item())
    return kayiplar
```

VERİMLİ

```
import torch


def egit(model, veri, kayip_fn, optimizer):
    kayiplar = []
    for x, y in veri:
        kayip = kayip_fn(model(x), y)
        kayip.backward()
        optimizer.step()
        kayiplar.append(kayip.detach())
    return torch.stack(kayiplar).tolist()
```

ÖLÇÜLMEDİ PyTorch ve bir GPU gerektirir. İşlemcide eşitleme beklemesi olmadığından fark yalnızca GPU'da görülür.

Kaynak: PyTorch Performance Tuning Guide · docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

---

YK090 orta Öneri verir

ölçülmedi

Döngüde aynı nesneyi cihaza taşıma

Döngü değişkenine bağlı olmayan bir nesne her adımda .to(device) ya da .cuda() ile yeniden taşınıyor. Bir kez, döngüden önce taşıyın.

NEDEN ENERJİ HARCAR?
Her taşıma CPU ile GPU arasında veri kopyalama ve eşitleme demektir.

NE ZAMAN SORUN DEĞİL?
Nesne her adımda farklıysa (yeni veri parçası) taşımak gerekir.

YAPILAN İŞ **n aktarım → 1**

İSRAFLI

```
def tahmin(model, veri, device):
    sonuclar = []
    for x in veri:
        model.to(device)
        sonuclar.append(model(x.to(device)))
    return sonuclar
```

VERİMLİ

```
def tahmin(model, veri, device):
    model.to(device)
    return [model(x.to(device)) for x in veri]
```

ÖLÇÜLMEDİ PyTorch ve bir GPU gerektirir. Kazanç, her adımda yapılan CPU→GPU kopyasının kalkmasıdır.

Kaynak: PyTorch Performance Tuning Guide · docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

---

YK091 orta Öneri verir ölçülmedi

Çıkarımda gradyan hesaplama

Model eval() moduna alınmış ama çağrılar torch.no_grad() ya da torch.inference_mode() içinde değil; gradyan için gereksiz bellek ve hesap harcanır.

NEDEN ENERJİ HARCAR?

Gradyan izleme her işlemin ara sonuçlarını geri yayılım için bellekte tutar; çıkarımda buna gerek yoktur.

NE ZAMAN SORUN DEĞİL?

Çıkarım sırasında gradyan gerekiyorsa (açıklanabilirlik, saldırı testleri) no_grad kullanılmaz.

YAPILAN İŞ **gradyan belleği → yok**

İSRAFLI

```
def degerlendir(model, veri):
    model.eval()
    dogru = 0
    for x, y in veri:
        cikti = model(x)
        dogru += int((cikti.argmax(1) == y).sum())
    return dogru
```

VERİMLİ

```
import torch


def degerlendir(model, veri):
    model.eval()
    dogru = 0
    with torch.inference_mode():
        for x, y in veri:
            cikti = model(x)
            dogru += int((cikti.argmax(1) == y).sum())
    return dogru
```

ÖLÇÜLMEDİ PyTorch gerektirir. Kazanç modele bağlıdır; çıkarımda gradyan için tutulan ara sonuçların belleği tamamen kalkar.

Kaynak: PyTorch Performance Tuning Guide: torch.no_grad · docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

07 Ağ ve bulut

7 kural

YK008 orta Öneri verir

ölçülmedi

Döngüde oturumsuz HTTP isteği

Her requests çağrısı yeni bir bağlantı ve TLS el sıkışması kurar. Döngüden önce requests.Session() açıp istekleri onunla gönderin; bağlantı yeniden kullanılır.

NEDEN ENERJİ HARCAR?

Oturumsuz her istek yeni bir TCP bağlantısı ve TLS el sıkışması kurar: istek başına birkaç gidiş-dönüş ve şifreleme hesabı. Session bağlantıyı açık tutar ve yeniden kullanır.

NE ZAMAN SORUN DEĞİL?

Her istek farklı bir sunucuya gidiyorsa bağlantı yeniden kullanılamaz.

YAPILAN İŞ **n el sıkışma → 1**

İSRAFLI

```
import requests


def fiyatlari_cek(urun_kodlari):
    return
  [requests.get(f"https://api.ornek.com.tr/fiyat/{kod}").json()
  for kod in urun_kodlari]
```

VERİMLİ

```
import requests


def fiyatlari_cek(urun_kodlari):
    with requests.Session() as oturum:
        return
  [oturum.get(f"https://api.ornek.com.tr/fiyat/{kod}",
  timeout=10).json() for kod in urun_kodlari]
```

ÖLÇÜLMEDİ Gerçek bir sunucuya HTTPS isteği gerektirir. Oturum, istek başına TCP ve TLS el sıkışmasını (çoğunlukla 1–2 gidiş-dönüş) ortadan kaldırır.

Kaynak: requests belgeleri: Session Objects (bağlantı havuzu) · `requests.readthedocs.io/en/latest/user/advanced/#session-objects`

YK014 yüksek Öneri verir

186× hızlı

Döngüde bağlantı, istemci ya da havuz oluşturma

Veritabanı bağlantısı, HTTP oturumu, bulut istemcisi ya da iş parçacığı havuzu kurmak pahalıdır (ağ el sıkışması, kimlik doğrulama). Döngüden önce bir kez oluşturup yeniden kullanın.

NEDEN ENERJİ HARCAR?

Bağlantı kurmak DNS, TCP, TLS ve kimlik doğrulama adımlarından geçer; çoğu zaman sorgunun kendisinden pahalıdır. Tek bağlantı ya da havuz bu bedeli bir kez öder.

NE ZAMAN SORUN DEĞİL?

Her adımda farklı bir hedefe bağlanılıyorsa yeniden kullanım mümkün değildir.

YAPILAN İŞ **n bağlantı → 1**

İSRAFLI

```
import sqlite3


def kayitlari_yaz(kayitlar):
    for k in kayitlar:
        baglanti = sqlite3.connect("veri.db")
        baglanti.execute("INSERT INTO log VALUES (?)", (k,))
        baglanti.commit()
        baglanti.close()
```

VERİMLİ

```
import sqlite3


def kayitlari_yaz(kayitlar):
    baglanti = sqlite3.connect("veri.db")
    with baglanti:
        baglanti.executemany("INSERT INTO log VALUES (?)",
  [(k,) for k in kayitlar])
    baglanti.close()
```

ÖLÇÜM süre 113,5 ms → 609 µs · bellek 3,8 KB → 4,9 KB · aynı sonuç ✓

Diskteki bir SQLite dosyasıyla ölçüldü. Her bağlantı kendi işlemini kapatmak zorunda olduğundan fark, commit sayısındaki azalmayı da içerir.

Kaynak: SQLAlchemy: Connection Pooling · requests: Session · `docs.sqlalchemy.org/en/20/core/pooling.html`

YK092 orta Öneri verir ölçülmedi

Döngüde tek tek nesne depolama isteği

Her get_object/head_object ayrı bir ağ isteğidir. İstekleri eşzamanlı yapın, listeleme sonucunu kullanın ya da veriyi toplu okuyun.

NEDEN ENERJİ HARCAR?

Her nesne isteği ayrı bir HTTP isteğidir; gecikme ve istek ücreti istek sayısıyla çarpılır.

NE ZAMAN SORUN DEĞİL?

Nesneler gerçekten ayrı ayrı gerekiyorsa istekleri eşzamanlı yapmak yine de büyük kazanç sağlar.

YAPILAN İŞ **n sıralı istek → eşzamanlı/toplu**

İSRAFLI

```
def raporlari_indir(s3, anahtarlar):
    return [s3.get_object(Bucket="raporlar", Key=k)
  ["Body"].read() for k in anahtarlar]
```

VERİMLİ

```
from concurrent.futures import ThreadPoolExecutor


def raporlari_indir(s3, anahtarlar):
    def indir(anahtar):
        return s3.get_object(Bucket="raporlar", Key=anahtar)
  ["Body"].read()

    with ThreadPoolExecutor(max_workers=16) as havuz:
        return list(havuz.map(indir, anahtarlar))
```

ÖLÇÜLMEDİ Bulut depolama (S3) hesabı gerektirir. Kazanç, sıralı istek gecikmelerinin üst üste binmesidir.

Kaynak: Green Software Foundation: Keep request counts low · AWS S3 performans kılavuzu · docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html

---

YK093 orta Öneri verir ölçülmedi

Redis'e döngüde tek tek komut

Her komut sunucuya ayrı bir gidiş-dönüştür. pipeline() ya da mget/mset ile komutları toplu gönderin.

NEDEN ENERJİ HARCAR?

Her komut bir ağ gidiş-dönüşüdür; pipeline komutları tek seferde gönderir ve yanıtları toplu alır.

NE ZAMAN SORUN DEĞİL?

Bir komutun sonucu sonrakini belirliyorsa pipeline kullanılamaz.

YAPILAN İŞ **n gidiş-dönüş → 1**

İSRAFLI

```
import redis


def sepetleri_getir(kullanicilar):
    r = redis.Redis()
    return [r.get(f"sepet:{k}") for k in kullanicilar]
```

VERİMLİ

```
import redis


def sepetleri_getir(kullanicilar):
    r = redis.Redis()
    return r.mget([f"sepet:{k}" for k in kullanicilar])
```

ÖLÇÜLMEDİ Redis sunucusu gerektirir. Kazanç gidiş-dönüş sayısındadır: n → 1.

Kaynak: Green Software Foundation: Keep request counts low · Redis: Pipelining · redis.io/docs/latest/develop/use/pipelining/

YK094 orta Öneri verir ölçülmedi

Döngüde urllib ile istek

urllib.request.urlopen her istekte yeni bir bağlantı açar. Bağlantıyı yeniden kullanan requests.Session ya da httpx.Client kullanın.

NEDEN ENERJİ HARCAR?

urllib her istekte yeni bağlantı açar ve kapatır; bağlantı havuzu yoktur.

NE ZAMAN SORUN DEĞİL?

Tek seferlik betiklerde fark önemsizdir.

YAPILAN İŞ **n bağlantı → 1**

İSRAFLI

```
import urllib.request


def sayfalari_indir(adresler):
    return [urllib.request.urlopen(a).read() for a in adresler]
```

VERİMLİ

```
import requests


def sayfalari_indir(adresler):
    with requests.Session() as oturum:
        return [oturum.get(a, timeout=10).content for a in
  adresler]
```

ÖLÇÜLMEDİ Gerçek bir sunucuya istek gerektirir. Kazanç, istek başına bağlantı kurulumunun kalkmasıdır.

Kaynak: requests belgeleri: Session Objects · requests.readthedocs.io/en/latest/user/advanced/#session-objects

---

YK095 orta Öneri verir ölçülmedi

async fonksiyonda senkron HTTP isteği

requests olay döngüsünü kilitler; bu sırada başka hiçbir iş ilerlemez. httpx.AsyncClient ya da aiohttp kullanın.

NEDEN ENERJİ HARCAR?
Olay döngüsü tek iş parçacığında döner; engelleyen istek bitene kadar diğer bütün görevler bekler.

NE ZAMAN SORUN DEĞİL?
İstek asyncio.to_thread ile ayrı iş parçacığına verilirse engelleme önlenir.

YAPILAN İŞ **seri bekleme → eşzamanlı**

İSRAFLI

```
import requests


async def hava_durumu(sehir):
    return requests.get(f"https://api.ornek.com/hava/{sehir}",
  timeout=5).json()
```

VERİMLİ

```
import httpx


async def hava_durumu(sehir):
    async with httpx.AsyncClient(timeout=5) as istemci:
        yanit = await
  istemci.get(f"https://api.ornek.com/hava/{sehir}")
        return yanit.json()
```

ÖLÇÜLMEDİ Gerçek bir sunucuya istek gerektirir. Kazanç tek çağrının süresinde değil, aynı anda bekleyen öbür görevlerin ilerleyebilmesindedir.

Kaynak: Python belgeleri: asyncio, engelleyen kod · docs.python.org/3/library/asyncio-dev.html#running-blocking-code

---

YK097 bilgi Öneri verir ölçülmedi

Zaman aşımı olmayan HTTP isteği

timeout verilmezse yanıt vermeyen bir sunucu isteği süresiz bekletir; süreç ve bağlantı boşuna açık kalır. timeout=... ekleyin.

NEDEN ENERJİ HARCAR?
Yanıt vermeyen bir sunucuya bağlı istek süresiz bekler; süreç, bağlantı ve bellek boşuna açık kalır.

NE ZAMAN SORUN DEĞİL?
Yok; her ağ isteğine bir zaman aşımı verin.

YAPILAN İŞ **süresiz → sınırlı**

İSRAFLI

```
import requests


def kur_getir():
    return requests.get("https://api.ornek.com/kur").json()
```

VERİMLİ

```
import requests


def kur_getir():
    return requests.get("https://api.ornek.com/kur",
  timeout=10).json()
```

ÖLÇÜLMEDİ Yanıt vermeyen bir sunucu gerektirir. Zaman aşımı, sonsuz beklemeyi belirli bir süreyle sınırlar.

Kaynak: requests belgeleri: Timeouts · requests.readthedocs.io/en/latest/user/advanced/#timeouts

---

08 Dosya ve G/Ç

8 kural

YK012 yüksek Öneri verir

ölçülmedi

Döngüde aynı dosyayı ya da modeli yeniden yükleme

Aynı dosya ya da model her döngü adımında diskten yeniden okunuyor. Döngüden önce bir kez yükleyip sonucu kullanın; disk ve işlemci yükü birlikte düşer.

NEDEN ENERJİ HARCAR?

Dosyayı ya da modeli her adımda diskten okuyup ayrıştırmak aynı sonucu tekrar üretir; büyük modellerde her yükleme saniyeler ve yüzlerce MB bellek demektir.

NE ZAMAN SORUN DEĞİL?

Dosya döngü sırasında gerçekten değişiyorsa (canlı izleme) yeniden okumak gerekir.

YAPILAN İŞ **n yükleme → 1**

İSRAFLI

```
import joblib


def tahmin_et(ornekler):
    sonuclar = []
    for x in ornekler:
        model = joblib.load("model.pkl")
        sonuclar.append(model.predict([x])[0])
    return sonuclar
```

VERİMLİ

```
import joblib


def tahmin_et(ornekler):
    model = joblib.load("model.pkl")
    return list(model.predict(ornekler))
```

ÖLÇÜLMEDİ Eğitilmiş bir model dosyası gerektirir. Kazanç, model boyutu × döngü uzunluğu kadar disk okuma ve ayrıştırmadır.

Kaynak: Green Software Foundation: Cache static data · patterns.greensoftware.foundation/development

---

YK019 bilgi Öneri verir

6,02× hızlı

Döngüde önceden biçimlendirilen log mesajı

f-string ya da format() ile hazırlanan log mesajı, o seviye kapalı olsa bile her adımda biçimlendirilir. logger.debug('%s', x) biçimini kullanın.

NEDEN ENERJİ HARCAR?

f-string, log seviyesi kapalı olsa bile çağrıdan önce değerlendirilir. %s biçiminde biçimlendirme yalnızca mesaj gerçekten yazılacaksa yapılır.

NE ZAMAN SORUN DEĞİL?

Mesaj her zaman yazılan bir seviyedeyse (error) fark yoktur.

YAPILAN İŞ **her adımda biçimlendirme → yalnızca gerekince**

İSRAFLI

```
import logging

logger = logging.getLogger(__name__)


def isle(kayitlar):
    for k in kayitlar:
        logger.debug(f"İşleniyor: {k}")
        k.kaydet()
```

VERİMLİ

```
import logging

logger = logging.getLogger(__name__)


def isle(kayitlar):
    for k in kayitlar:
        logger.debug("İşleniyor: %s", k)
        k.kaydet()
```

ÖLÇÜM süre 3,6 ms → 598 µs · bellek 442 B → 48 B · aynı sonuç ✓

Günlük seviyesi WARNING; debug mesajları yazılmıyor ama kötü sürüm yine de her birini biçimlendiriyor.

Kaynak: Pylint W1203 logging-fstring-interpolation · docs.python.org/3/howto/logging.html#optimization

---

YK020 bilgi Öneri verir

132× az bellek

Dosyayı readlines() ile belleğe alma

readlines() bütün dosyayı önce belleğe okur. Satırları doğrudan dosya nesnesi üzerinde gezin: for satir in dosya.

NEDEN ENERJİ HARCAR?

readlines bütün dosyayı bir kerede belleğe alır ve satır listesi kurar; bellek dosya boyutu kadar büyür. Dosya nesnesi tamponla satır satır okur.

NE ZAMAN SORUN DEĞİL?

Satırlara rastgele erişim ya da birden fazla geçiş gerekiyorsa liste gerekebilir.

YAPILAN İŞ **O(dosya) bellek → O(satır)**

İSRAFLI

```
def hata_say(log_yolu):
    sayi = 0
    with open(log_yolu) as f:
        for satir in f.readlines():
            if "ERROR" in satir:
                sayi += 1
    return sayi
```

VERİMLİ

```
def hata_say(log_yolu):
    with open(log_yolu) as f:
        return sum(1 for satir in f if "ERROR" in satir)
```

ÖLÇÜM süre 6,1 ms → 5,6 ms · bellek 5,9 MB → 45,6 KB · aynı sonuç ✓

Süre benzer; asıl kazanç bellektedir: iyi sürüm dosyayı satır satır okur.

Kaynak: Ruff FURB129 readlines-in-for · docs.astral.sh/ruff/rules/readlines-in-for/

YK098 orta Öneri verir

27,6× hızlı

Döngüde flush()

flush() tamponu her adımda boşaltıp ayrı bir sistem çağrısı yapar. Tamponlamayı işletim sistemine bırakın, gerekirse döngüden sonra bir kez flush edin.

NEDEN ENERJİ HARCAR?

Tamponlu yazma birçok küçük yazmayı tek sistem çağrısında birleştirir; flush bunu her adımda bozar.

NE ZAMAN SORUN DEĞİL?

Çıktının anında görünmesi gerekiyorsa (canlı log) arada bir flush yeterlidir.

YAPILAN İŞ **n sistem çağrısı → az**

İSRAFLI

```
def kaydet(dosya, satirlar):
    for s in satirlar:
        dosya.write(s + "\n")
        dosya.flush()
```

VERİMLİ

```
def kaydet(dosya, satirlar):
    dosya.writelines(s + "\n" for s in satirlar)
```

ÖLÇÜM süre 12,6 ms → 457 µs · bellek 360 B → 46,8 KB · aynı sonuç ✓

Kaynak: Python belgeleri: io (tamponlu akışlar) · docs.python.org/3/library/io.html

---

YK099 yüksek Öneri verir

41,4× hızlı

Döngüde os.fsync()

fsync diskin yazmayı fiziksel olarak bitirmesini bekler; döngüde çok pahalıdır. Toplu yazıp bir kez eşitleyin.

NEDEN ENERJİ HARCAR?

fsync diskin verinin kalıcı olduğunu onaylamasını bekler; SSD'de bile milisaniyeler sürer.

NE ZAMAN SORUN DEĞİL?

Her kaydın tek tek kalıcı olması gerekiyorsa (işlem günlüğü) gereklidir; toplu yazıp bir kez eşitlemek çoğu zaman yeter.

YAPILAN İŞ **n disk eşitleme → 1**

İSRAFLI

```
import os


def guvenli_yaz(f, kayitlar):
    for k in kayitlar:
        f.write(k)
        f.flush()
        os.fsync(f.fileno())
```

VERİMLİ

```
import os


def guvenli_yaz(f, kayitlar):
    f.writelines(kayitlar)
    f.flush()
    os.fsync(f.fileno())
```

ÖLÇÜM süre 2,8 ms → 68 µs · bellek 360 B → 6,0 KB · aynı sonuç ✓

Kaynak: SQLite SSS ve POSIX: fsync maliyeti · www.sqlite.org/faq.html#q19

YK100 bilgi Öneri verir 3,56× hızlı

Döngüde print()

print() her çağrıda çıktı akışına yazar; büyük döngülerde ciddi G/Ç yüküdür. Sonuçları toplayıp bir kez yazdırın ya da logging seviyelerini kullanın.

NEDEN ENERJİ HARCAR?
Her print bir biçimlendirme ve yazmadır; terminale yazmak özellikle yavaştır ve çoğu zaman satır satır boşaltılır.

NE ZAMAN SORUN DEĞİL?
Kısa betiklerde ve geliştirme sırasında sorun değildir.

YAPILAN İŞ **n yazma → 1**

İSRAFLI

```
def isle(kayitlar):
    for k in kayitlar:
        print("İşleniyor:", k)
```

VERİMLİ

```
def isle(kayitlar):
    print("\n".join(f"İşleniyor: {k}" for k in kayitlar))
```

ÖLÇÜM süre 1,4 ms → 399 µs · bellek 252,2 KB → 740,1 KB · aynı sonuç ✓

Çıktı bellekteki bir tampona yönlendirildi. Terminale ya da dosyaya yazarken her çağrı daha pahalıdır.

Kaynak: Python belgeleri: logging HOWTO · docs.python.org/3/howto/logging.html

---

YK101 orta Öneri verir 280× hızlı

Döngüde dizini yeniden tarama

os.listdir/glob aynı dizini her adımda diskte yeniden tarar. Sonucu döngüden önce bir kez alın.

NEDEN ENERJİ HARCAR?
Dizin listelemek dosya sistemi meta verisini okur; aynı dizini her adımda taramak aynı sonucu tekrar üretir.

NE ZAMAN SORUN DEĞİL?
Dizin döngü sırasında değişiyorsa (yeni dosyalar bekleniyorsa) yeniden taramak gerekir.

YAPILAN İŞ **n tarama → 1**

İSRAFLI

```
import os


def yeni_dosyalar(gorulen):
    yeniler = []
    for g in gorulen:
        if g not in os.listdir("gelen"):
            yeniler.append(g)
    return yeniler
```

VERİMLİ

```
import os


def yeni_dosyalar(gorulen):
    mevcut = set(os.listdir("gelen"))
    return [g for g in gorulen if g not in mevcut]
```

ÖLÇÜM süre 56,8 ms → 203 µs · bellek 19,3 KB → 28,1 KB · aynı sonuç ✓

Kaynak: Python belgeleri: os.scandir · docs.python.org/3/library/os.html#os.scandir

---

YK105 bilgi Öneri verir fark yok

Döngüde tek tek write()

Her write() ayrı bir çağrıdır. f.writelines(satirlar) aynı işi tek çağrıda yapar.

NEDEN ENERJİ HARCAR?
Her write bir metot çağrısıdır; writelines yinelenebiliri tek çağrıda yazar.

NE ZAMAN SORUN DEĞİL?
Satırlar arasında başka iş yapılıyorsa döngü gerekebilir.

YAPILAN İŞ **n çağrı → 1**

İSRAFLI

```
def kaydet(yol, satirlar):
    with open(yol, "w") as f:
        for s in satirlar:
            f.write(s)
```

VERİMLİ

```
def kaydet(yol, satirlar):
    with open(yol, "w") as f:
        f.writelines(satirlar)
```

ÖLÇÜM süre 2,0 ms → 2,2 ms · bellek 51,2 KB → 51,2 KB · aynı sonuç ✓

Tamponlu dosyada fark yok: write çağrıları zaten bellekteki tampona yazar. Kazanç sadeliktedir.

Kaynak: Ruff FURB122 for-loop-writes · docs.astral.sh/ruff/rules/for-loop-writes/

---

09 Eşzamanlılık ve bekleme

7 kural

YK005 orta Öneri verir

ölçülmedi

Sürekli sorgulama (polling)

Sonsuz döngüde bekleyip tekrar kontrol etmek işlemciyi ve ağı boşuna meşgul eder. Olay tabanlı yapı (webhook, kuyruk, bildirim) kullanın ya da bekleme süresini artırın.

NEDEN ENERJİ HARCAR?

Sürekli kontrol eden döngü, ortada iş yokken de işlemciyi uyandırır ve ağ isteği gönderir; karşı tarafta da boşa yük oluşturur.

NE ZAMAN SORUN DEĞİL?

Olay mekanizması olmayan eski sistemlerde uzun aralıklı (dakikalar) sorgulama kabul edilebilir.

YAPILAN İŞ **sürekli uyanma → yalnızca olay gelince**

İSRAFLI

```
import time


def siparis_bekle(api):
    while True:
        if api.yeni_siparis_var_mi():
            return api.siparisi_al()
        time.sleep(0.1)
```

VERİMLİ

```
import queue


def siparis_bekle(siparisler: queue.Queue):
    # Webhook ya da kuyruk: sipariş gelince uyanır, arada
  işlemci ve ağ boştadır.
    return siparisler.get()
```

ÖLÇÜLMEDİ Dış bir servisi (API) bekler. Kazanç istek sayısındadır: 0,1 saniyede bir sorgulayan döngü bir dakikada 600 istek gönderir, olay tabanlı yapı yalnızca bir.

Kaynak: Green Software Foundation: Use Asynchronous network calls instead of synchronous · patterns.greensoftware.foundation/development

---

YK015 yüksek Öneri verir

544× az işlemci

Boş döngüyle bekleme (busy wait)

Gövdesi boş bir döngü, koşul değişene kadar bir işlemci çekirdeğini %100 meşgul eder. threading.Event, kuyruk ya da time.sleep ile bekleyin.

NEDEN ENERJİ HARCAR?

Boş döngü işlemciyi hiç uyutmaz; çekirdek koşul değişene kadar tam güçte döner. Bu, doğrudan pil, ısı ve elektrik olarak harcanır.

NE ZAMAN SORUN DEĞİL?

Mikrosaniyelik düşük seviye eşzamanlılık kodunda bilinçli olarak kullanılabilir; uygulama kodunda neredeyse hiç.

YAPILAN İŞ **%100 çekirdek → ~%0**

İSRAFLI

```
def hazir_olmasini_bekle(is_parcacigi):
    while not is_parcacigi.bitti:
        pass
    return is_parcacigi.sonuc
```

VERİMLİ

```
def hazir_olmasini_bekle(is_parcacigi):
    is_parcacigi.join()
    return is_parcacigi.sonuc
```

ÖLÇÜM süre 31,3 ms → 25,0 ms · işlemci 31,3 ms → 58 µs · bellek 144 B → 144 B · aynı sonuç ✓

İki sürüm de işin bitmesini 20 ms bekler; süre aynıdır. Fark işlemcinin bu sürede yaptığı boş iştedir, bu yüzden oran işlemci süresinden hesaplandı.

Kaynak: Python belgeleri: threading.Event · docs.python.org/3/library/threading.html#event-objects

---

YK030 orta Öneri verir ölçülmedi

Döngüde süreç başlatma

Her subprocess ya da os.system çağrısı yeni bir işletim sistemi süreci başlatır. İşi toplu yaptırın ya da aynı işi gören Python kütüphanesini kullanın.

NEDEN ENERJİ HARCAR?

Her süreç başlatma işletim sisteminde fork/exec, bellek eşleme ve program yükleme demektir; kısa işler için çoğu zaman asıl işten pahalıdır.

NE ZAMAN SORUN DEĞİL?

Dış programın Python karşılığı yoksa ve iş toplu verilemiyorsa kaçınılmazdır.

YAPILAN İŞ **n süreç → 1**

İSRAFLI

```
import subprocess


def resimleri_kucult(dosyalar):
    for d in dosyalar:
        subprocess.run(["convert", d, "-resize", "50%",
  "kucuk_" + d])
```

VERİMLİ

```
import subprocess


def resimleri_kucult(dosyalar):
    # Tek süreç bütün dosyaları işler (ya da Pillow ile hiç
  süreç açmadan).
    subprocess.run(["mogrify", "-resize", "50%", "-path",
  "kucuk", *dosyalar], check=True)
```

ÖLÇÜLMEDİ Dış bir program (ImageMagick) gerektirir. Her süreç başlatma milisaniyeler sürer; tek süreç bunu bir kez öder.

Kaynak: Python belgeleri: subprocess · docs.python.org/3/library/subprocess.html

---

YK096 orta Öneri verir ölçülmedi

async fonksiyonda time.sleep

time.sleep olay döngüsünü tamamen durdurur. await asyncio.sleep(...) kullanın.

NEDEN ENERJİ HARCAR?

time.sleep iş parçacığını uyutur; olay döngüsü de onunla birlikte durur.

NE ZAMAN SORUN DEĞİL?

Yok; async kodda her zaman asyncio.sleep kullanın.

YAPILAN İŞ **döngü durur → döngü çalışır**

İSRAFLI

```
import time


async def tekrar_dene(istek):
    for _ in range(3):
        if await istek():
            return True
        time.sleep(1)
    return False
```

VERİMLİ

```
import asyncio


async def tekrar_dene(istek):
    for _ in range(3):
        if await istek():
            return True
        await asyncio.sleep(1)
    return False
```

ÖLÇÜLMEDİ Kazanç tek çağrının süresinde değil, olay döngüsündeki öbür görevlerin bekleme sırasında ilerleyebilmesindedir.

Kaynak: Python belgeleri: asyncio.sleep · docs.python.org/3/library/asyncio-task.html#asyncio.sleep

---

YK102 orta Öneri verir

3,26× az işlemci

Çok kısa aralıklarla uyanma

Döngüde çok kısa time.sleep işlemcinin derin uyku durumlarına geçmesini engeller; saniyede onlarca kez uyanır. Bekleme süresini artırın ya da olay tabanlı bekleyin.

NEDEN ENERJİ HARCAR?

Her uyanma işlemciyi derin uyku durumundan çıkarır; saniyede onlarca uyanma, işlemcinin düşük güç durumlarına geçmesini engeller.

NE ZAMAN SORUN DEĞİL?

Gerçek zamanlı denetim gibi kısa gecikmenin şart olduğu durumlarda gerekebilir.

YAPILAN İŞ **çok uyanma → az uyanma**

İSRAFLI

```
import time


def kuyruk_bekle(kuyruk):
    while kuyruk.bos_mu():
        time.sleep(0.001)
    return kuyruk.al()
```

VERİMLİ

```
def kuyruk_bekle(kuyruk):
    return kuyruk.get()
```

ÖLÇÜM süre 55,2 ms → 52,9 ms · işlemci 805 µs → 247 µs · bellek 144 B → 264 B · aynı sonuç ✓

İki sürüm de 50 ms sonra gelen işi bekler; süre aynıdır. Kötü sürüm bu sürede yaklaşık 50 kez uyanır; oran işlemci süresinden hesaplandı.

Kaynak: Linux powertop / Intel: zamanlayıcı uyanmaları · `github.com/fenrus75/powertop`

YK103 bilgi Öneri verir

1,23× hızlı

shell=True ile süreç başlatma

shell=True komutu çalıştırmak için ek bir kabuk süreci başlatır. Komutu liste olarak verip shell=False kullanın.

NEDEN ENERJİ HARCAR?

shell=True önce /bin/sh başlatır, komutu kabukta ayrıştırır, sonra asıl programı başlatır: fazladan bir süreç.

NE ZAMAN SORUN DEĞİL?

Boru (|) gibi kabuk özellikleri gerçekten gerekiyorsa kullanılabilir; girdiler güvenilir olmalıdır.

YAPILAN İŞ **2 süreç → 1**

İSRAFLI

```
import subprocess


def disk_kullanimi():
    return subprocess.run("df -h", shell=True,
  capture_output=True).stdout
```

VERİMLİ

```
import subprocess


def disk_kullanimi():
    return subprocess.run(["df", "-h"],
  capture_output=True).stdout
```

ÖLÇÜM süre 14,1 ms → 11,5 ms · bellek 82,5 KB → 84,4 KB · aynı sonuç ✓

Süre farkı, fazladan başlatılan kabuk sürecinden gelir.

Kaynak: Python belgeleri: subprocess (güvenlik ve shell=True) · `docs.python.org/3/library/subprocess.html#security-considerations`

YK104 bilgi Öneri verir

ölçülmedi

Hata ayıklama modunda sunucu

debug=True otomatik yeniden yükleyici için ikinci bir süreç çalıştırır ve dosyaları sürekli izler. Üretimde debug=False kullanın.

NEDEN ENERJİ HARCAR?

Yeniden yükleyici ikinci bir süreç çalıştırır ve dosyaları düzenli aralıklarla tarar; hata ayıklayıcı ek iş yapar.

NE ZAMAN SORUN DEĞİL?

Geliştirme sırasında uygundur; üretimde kapatılmalıdır.

YAPILAN İŞ **2 süreç + dosya tarama → 1 süreç**

İSRAFLI

```
from flask import Flask

app = Flask(__name__)

if __name__ == "__main__":
    app.run(debug=True)
```

VERİMLİ

```
import os

from flask import Flask

app = Flask(__name__)

if __name__ == "__main__":
    app.run(debug=os.getenv("FLASK_DEBUG") == "1")
```

ÖLÇÜLMEDİ Sürekli çalışan bir sunucudur. Kazanç, ikinci süreç ve dosya izleme yükünün kalkmasıdır.

Kaynak: Flask belgeleri: Debug Mode · `flask.palletsprojects.com/en/stable/quickstart/#debug-mode`

Ölçüm yöntemi ve sınırlar

- Her kuralın israflı (`ornekler/kurallar/YKxxx.py`) ve verimli (`iyi/YKxxx.py`) örneği aynı adla ve aynı girdiyle çalıştırılır. Girdi her iki sürüm için aynı tohumla üretilir.
- Önce çıktılar karşılaştırılır: dönüş değeri ya da bıraktığı iz (veritabanı, dosya, ekran çıktısı). Aynı değilse ölçüm yapılmaz.
- Süre ve işlemci süresi iki sürüm sırayla (A, B, A, B…) ölçülerek alınır; makinenin anlık yükü iki tarafı eşit etkiler. Her tur yaklaşık 60 ms sürer; sonuç 5 turun ortancasıdır.
- Girdisini değiştiren örneklerde her çağrıdan önce girdi yeniden kurulur ve kurulum ölçüme girmez. Önbellekler her çağrıdan önce temizlenir.
- Bellek tepe değeri ayrı bir çalıştırmada tracemalloc ile ölçülür; yalnızca çağrı sırasında ayrılan bellek sayılır.
- Veritabanı kuralları bellekteki (bağlantı ve commit kuralları diskteki) SQLite ile ölçüldü. Ağ üzerindeki bir veritabanında her sorgu ayrıca bir gidiş-dönüş ekler; gerçek fark genellikle daha büyüktür.
- Herkes tekrarlayabilir: `python -m greencode kanit` bütün ölçümleri yeniden alır. Sitede her kuralın sayfasından aynı ölçüm tarayıcıda yapılabilir.

Hızda kazanç göstermeyen kurallar

Bu kuralların değeri bellekte, doğrulukta ya da okunabilirliktedir. Ölçüm bunu gizlemeden gösteriyor.

YK020	Dosyayı readlines() ile belleğe alma	132× az bellek	Süre benzer; asıl kazanç bellektedir: iyi sürüm dosyayı satır satır okur.
YK036	Tek satır için LIMIT olmadan sorgu	fark yok	SQLite imleci zaten ilk satırda durur; bu yüzden fark yok denecek kadar azdır. Fark, sonucu önceden hazırlayıp ağdan gönderen sunucu veritabanlarında görülür.
YK049	Döngü içinde try/except	1,15× yavaş	Python 3.11'den beri try bloğuna girmenin maliyeti yok; bu ölçümün yapıldığı sürümde fark beklenmez.
YK052	Elle sözlük kurgusu	fark yok	
YK062	Gereksiz çift dönüşüm	fark yok	Fark ölçülemeyecek kadar küçük: fazladan kopya C ile yapılır ve sıralamanın yanında önemsizdir. Kazanç sadeliktedir.
YK063	reversed(sorted(...)) ya da sorted(...)[::-1]	fark yok	Fark ölçülemeyecek kadar küçük: reversed() kopyası sıralamanın yanında önemsizdir. reverse=True ayrıca eşit elemanların sırasını korur.
YK065	sum(), min() ya da max() içinde liste	1.089× az bellek	Üreteç bellekte yer tutmaz ama CPython'da liste kurgusundan biraz yavaş çalışabilir; kazanç bellektedir.
YK067	Saymak için len(list(...))	216× az bellek	Süre benzer; asıl kazanç bellektedir: satırlar sayılırken listede tutulmaz.
YK069	Düz metin için düzenli ifade	fark yok	
YK070	Döngüde derlenmemiş düzenli ifade	fark yok	
YK072	İçerme kontrolü için find()	fark yok	Hız aynı: ikisi de aynı C aramasını yapar. Kazanç okunabilirliktedir; Python belgeleri de içerme için 'in' önerir.
YK073	Bütün dosyayı okuyup satırlara bölme	215× az bellek	Süre benzer; asıl kazanç bellektedir: iyi sürüm dosyayı satır satır okur.
YK082	Zincirleme indeksleme ile atama	1,20× yavaş	Bu kuralın asıl amacı doğruluktur: zincirleme atama kopya üzerinde çalışırsa değişiklik kaybolur. pandas 3.0'da (copy-on-write) kötü sürüm tabloyu hiç değiştirmez.
YK105	Döngüde tek tek write()	fark yok	Tamponlu dosyada fark yok: write çağrıları zaten bellekteki tampona yazar. Kazanç sadeliktedir.
YK111	Sabitlik kontrolü için nunique()	28,2× az bellek	Bütün değerler aynı olduğu için iki sürüm de seriyi sonuna kadar tarar ve süre aynıdır. Kazanç bellektedir: nunique bütün değerlerin karma tablosunu kurar.

Ölçülmeyen kurallar ve nedenleri

YK005	Sürekli sorgulama (polling)	Dış bir servisi (API) bekler. Kazanç istek sayısındadır: 0,1 saniyede bir sorgulayan döngü bir dakikada 600 istek gönderir, olay tabanlı yapı yalnızca bir.
YK008	Döngüde oturumsuz HTTP isteği	Gerçek bir sunucuya HTTPS isteği gerektirir. Oturum, istek başına TCP ve TLS el sıkışmasını (çoğunlukla 1–2 gidiş-dönüş) ortadan kaldırır.
YK012	Döngüde aynı dosyayı ya da modeli yeniden yükleme	Eğitilmiş bir model dosyası gerektirir. Kazanç, model boyutu × döngü uzunluğu kadar disk okuma ve ayrıştırmadır.
YK030	Döngüde süreç başlatma	Dış bir program (ImageMagick) gerektirir. Her süreç başlatma milisaniyeler sürer; tek süreç bunu bir kez öder.
YK033	Döngüde objects.create() (Django)	Django projesi ve veritabanı sunucusu gerektirir. Kazanç sorgu sayısındadır: n INSERT → 1.

YK034	**Döngüde save()**	Django projesi ve veritabanı sunucusu gerektirir. Kazanç sorgu sayısındadır: n UPDATE → 1.
YK035	**Saymak için sorgu sonucunu len() ile ölçme**	Django projesi ve veritabanı sunucusu gerektirir. Kazanç taşınan veridedir: n nesne → 1 sayı.
YK075	**Döngüde DataFrame.append**	DataFrame.append pandas 2.0'da kaldırıldı; kötü sürüm güncel pandas'ta çalışmaz. Aynı kalıbın çalışan hâli YK076'da ölçüldü.
YK089	**Döngüde GPU'dan değer çekme**	PyTorch ve bir GPU gerektirir. İşlemcide eşitleme beklemesi olmadığından fark yalnızca GPU'da görülür.
YK090	**Döngüde aynı nesneyi cihaza taşıma**	PyTorch ve bir GPU gerektirir. Kazanç, her adımda yapılan CPU→GPU kopyasının kalkmasıdır.
YK091	**Çıkarımda gradyan hesaplama**	PyTorch gerektirir. Kazanç modele bağlıdır; çıkarımda gradyan için tutulan ara sonuçların belleği tamamen kalkar.
YK092	**Döngüde tek tek nesne depolama isteği**	Bulut depolama (S3) hesabı gerektirir. Kazanç, sıralı istek gecikmelerinin üst üste binmesidir.
YK093	**Redis'e döngüde tek tek komut**	Redis sunucusu gerektirir. Kazanç gidiş-dönüş sayısındadır: n → 1.
YK094	**Döngüde urllib ile istek**	Gerçek bir sunucuya istek gerektirir. Kazanç, istek başına bağlantı kurulumunun kalkmasıdır.
YK095	**async fonksiyonda senkron HTTP isteği**	Gerçek bir sunucuya istek gerektirir. Kazanç tek çağrının süresinde değil, aynı anda bekleyen öbür görevlerin ilerleyebilmesindedir.
YK096	**async fonksiyonda time.sleep**	Kazanç tek çağrının süresinde değil, olay döngüsündeki öbür görevlerin bekleme sırasında ilerleyebilmesindedir.
YK097	**Zaman aşımı olmayan HTTP isteği**	Yanıt vermeyen bir sunucu gerektirir. Zaman aşımı, sonsuz beklemeyi belirli bir süreyle sınırlar.
YK104	**Hata ayıklama modunda sunucu**	Sürekli çalışan bir sunucudur. Kazanç, ikinci süreç ve dosya izleme yükünün kalkmasıdır.